Práctica: Autoencoders Variacionales

VAE y WAE — Aprendizaje con Espacio Latente

Redes Neuronales y Aprendizaje Profundo
Grado en Ingeniería en Inteligencia Artificial
Curso: 2025/2026

Autores:
Jesús Cano Moya
Izan Cuesta Corbí
Dennis García Solera
Marcos Segurado Llopis

Universidad de Alicante

Índice general

1. Introducción **3**

2. Configuración global **4**
2.1. Funciones de pérdida 4
2.1.1. VAE clásico: pérdida KL 4
2.1.2. WAE: pérdida MMD 5
2.2. Técnicas de regularización del entrenamiento 7
2.2.1. KL Annealing en el entorno 2D 7
2.2.2. Dropout y Early Stopping en el entorno 3D 7
2.3. Arquitecturas encoder-decoder 8
2.3.1. Diferencia entre VAE y WAE en la arquitectura 2D 8
2.3.2. Adaptación a múltiples datasets 2D 9
2.3.3. Arquitectura Convolucional base para Volúmenes 3D 10
2.4. Bucle de entrenamiento y encapsulación 11
2.4.1. Función de entrenamiento unificada 2D 11
2.4.2. Encapsulación e inferencia integrada en Keras (3D) 11
2.5. Funciones de visualización e interpolación 11
2.5.1. Espacio latente en entornos 2D y 3D 11
2.5.2. Interpolación en el espacio latente 12

3. Apartado 2D: VAE y WAE en imágenes 2D **13**
3.1. Hiperparámetros específicos del apartado 2D 13
3.2. MNIST 13
3.2.1. VAE clásico (KL) 13
3.2.2. WAE (MMD) 16
3.2.3. Evolución del entrenamiento 19
3.2.4. Exploración de valores de peso 20
3.2.5. Análisis de la representación en MNIST 20
3.3. FashionMNIST 22
3.3.1. VAE clásico (KL) 22
3.3.2. WAE (MMD) 23
3.3.3. Evolución del entrenamiento 25
3.3.4. Análisis de la representación en FashionMNIST 25
3.4. CIFAR-10 26
3.4.1. VAE clásico (KL) 26
3.4.2. WAE (MMD) 27
3.4.3. Evolución del entrenamiento 29

3.4.4. Exploración del espacio de dimensiones 29
3.4.5. Análisis de la representación en CIFAR-10 29
3.5. Conclusiones del Apartado 2D 30

4. Apartado 3D: Autoencoders en Volúmenes Tridimensionales 32
4.1. Hiperparámetros específicos del apartado 3D 32
4.2. 3D MNIST (Dataset de Números en 3D) 33
4.2.1. VAE clásico 3D (KL) 33
4.2.2. WAE 3D (MMD) 35
4.2.3. Evolución del entrenamiento 37
4.2.4. Análisis comparativo de la representación en 3D MNIST 38
4.3. ModelNet (Subconjunto de 4 clases) 38
4.3.1. El reto de la dimensionalidad: Overfitting y Regularización . . . 39
4.3.2. Resultados del VAE clásico 3D (KL) 39
4.3.3. Resultados del WAE 3D (MMD) Definitivo 43
4.3.4. Análisis comparativo de la representación en ModelNet (KL vs MMD) 46

5. Conclusiones Generales 47

Capítulo 1

Introducción

La realización de este proyecto se basa en los modelos generativos basados en espacio latente, concretamente sobre los *Variational Autoencoders* (VAE) y los *Wasserstein Autoencoders* (WAE).

La práctica se divide en dos grandes bloques:

- **Apartado 2D:** Implementación y evaluación de VAE con función de pérdida KL clásica y WAE con función de pérdida MMD, aplicados sobre los conjuntos de datos MNIST, FashionMNIST y CIFAR-10.
- **Apartado 3D:** Implementación y evaluación de autoencoders probabilísticos en tensores volumétricos, adaptando las arquitecturas para procesar datos espaciales 3D como el dataset 3D MNIST y el conjunto de polígonos ModelNet.

El objetivo general es comparar ambas formulaciones de la regularización del espacio latente, analizar su comportamiento en diferentes dominios (imagen plana vs volumen tridimensional) y estudiar propiedades como la calidad de reconstrucción, la estructura del espacio latente y la capacidad generativa.

Capítulo 2

Configuración global

En este apartado, describimos los componentes de código que son comunes a todos los apartados de la práctica: las funciones de pérdida, las técnicas de regularización del entrenamiento, las arquitecturas encoder-decoder y las utilidades de visualización e interpolación. Tanto el Apartado 2D como el 3D se apoyan en estos bloques, mientras los hiperparámetros específicos de cada dataset se detallan dentro de su apartado correspondiente.

2.1. Funciones de pérdida

2.1.1. VAE clásico: pérdida KL

En el VAE estándar, la función de pérdida combina el error de reconstrucción (MSE o BCE) con la divergencia KL entre la distribución aprendida $q_\phi(z|x)$ y la prior $p(z) = \mathcal{N}(0, I)$:

$$\mathcal{L}_{\text{KL}} = \underbrace{\|x - \hat{x}\|^2}_{\text{reconstrucción}} + \beta \cdot \underbrace{\text{KL}\big(q_\phi(z|x) \,\|\, p(z)\big)}_{\text{regularización}} \tag{2.1}$$

donde $\beta = 1$ en la configuración estándar. El término KL penaliza cualquier desviación de la distribución posterior respecto a la prior gaussiana, lo que fuerza al espacio latente a ser continuo y bien organizado. Su expresión analítica, derivada de la suposición de distribuciones gaussianas diagonales, es:

$$\text{KL} = -\frac{1}{2}\,\mathbb{E}\Big[\sum_j \big(1 + \log \sigma_j^2 - \mu_j^2 - \sigma_j^2\big)\Big] \tag{2.2}$$

Implementación en 2D (PyTorch)

En la implementación 2D se calcula con `torch.sum` por dimensión latente y `torch.mean` sobre el batch, de modo que el gradiente escala correctamente con el tamaño del lote:

Listing 2.1: Función de pérdida KL en PyTorch (2D).

```python
def vae_loss_kl(recon_x, x, mu, logvar, beta=1.0):
    recon = F.mse_loss(recon_x, x, reduction="sum")
    kl = -0.5 * torch.mean(torch.sum(1 + logvar - mu.pow(2)
                                     - logvar.exp(), dim=1))
    return recon + beta * kl, recon, kl
```

Implementación en 3D (TensorFlow / Keras)

Para el dominio tridimensional, la pérdida de reconstrucción se adapta utilizando la entropía cruzada binaria (`binary_crossentropy`) extendida sobre las dimensiones espaciales del cubo ($32 \times 32 \times 32$). El término de regularización KL mantiene su formulación matemática, pero se reduce la influencia de su gradiente mediante un factor $\beta = 0{,}01$ adaptado para equilibrar la magnitud del error frente a la alta densidad de vóxeles:

Listing 2.2: Cálculo de pérdidas VAE en TensorFlow (3D).

```python
# Dentro del paso de entrenamiento del modelo 3D:
recon_loss = tf.reduce_mean(
    tf.reduce_sum(tf.keras.losses.binary_crossentropy(data,
        reconstruccion), axis=(1, 2, 3))
)

kl_loss = -0.5 * (1 + z_log_var - tf.square(z_mean) - tf.exp(z_log_var
    ))
kl_loss = tf.reduce_mean(tf.reduce_sum(kl_loss, axis=1))

total_loss = recon_loss + (self.beta * kl_loss)
```

2.1.2. WAE: pérdida MMD

En la variante WAE [2] la regularización KL se sustituye por la *Maximum Mean Discrepancy* (MMD), que mide la distancia entre la distribución agregada del encoder $q(z) = \int q_\phi(z|x)\, p(x)\, dx$ y la prior $p(z)$, sin necesidad de calcular una KL analítica:

$$\mathcal{L}_{\text{MMD}} = \|x - \hat{x}\|^2 + \lambda \cdot \widehat{\text{MMD}}^2\big(q(z),\, p(z)\big) \tag{2.3}$$

La estimación empírica de la MMD a partir de muestras se descompone en tres términos de kernel:

$$\widehat{\text{MMD}}^2(q, p) = \mathbb{E}_{z,z' \sim q}[k(z, z')] + \mathbb{E}_{u,u' \sim p}[k(u, u')] - 2\,\mathbb{E}_{z \sim q,\, u \sim p}[k(z, u)] \tag{2.4}$$

Elección del kernel: IMQ frente a RBF

La implementación utiliza el kernel *Inverse Multi-Quadratic* (IMQ) en lugar del kernel gaussiano (RBF):

$$k_{\text{IMQ}}(x, y) = \frac{C}{C + \|x - y\|^2}, \qquad C = 2 \cdot d \cdot 2 \tag{2.5}$$

donde d es la dimensión del espacio latente. La constante C se fija en $4d$ de acuerdo con la propuesta original del WAE para escalar adecuadamente con la dimensionalidad.

La razón de preferir IMQ sobre RBF es su comportamiento de cola gruesa: el kernel RBF decae exponencialmente y prácticamente se anula para puntos moderadamente distantes, lo que puede producir gradientes muy pequeños y dificultar el entrenamiento. El kernel IMQ, al decaer polinomialmente, mantiene señal de gradiente incluso cuando z y z' están lejos, lo que lo hace más robusto en espacios latentes de alta dimensión.

Implementación de la MMD

El cálculo se divide en tres funciones para mayor claridad:

Listing 2.3: Estimación de la MMD con kernel IMQ.

```
def compute_kernel(x, y):
    x_size = x.shape[0]
    y_size = y.shape[0]
    dim    = x.shape[1]

    # Expandimos para calcular todas las combinaciones de pares
    x = x.unsqueeze(1)                          # (N, 1, d)
    y = y.unsqueeze(0)                          # (1, M, d)
    tiled_x = x.expand(x_size, y_size, dim)
    tiled_y = y.expand(x_size, y_size, dim)

    C = 2.0 * dim * 2.0                         # constante del kernel
        IMQ
    kernel = C / (1e-7 + C + (tiled_x - tiled_y).pow(2).sum(dim=-1))
    return kernel                               # (N, M)

def compute_mmd(z, z_prior):
    xx = compute_kernel(z,       z)
    yy = compute_kernel(z_prior, z_prior)
    xy = compute_kernel(z,       z_prior)

    N = xx.shape[0]
    # Excluimos la diagonal (distancia de un punto consigo mismo)
    xx_val = (xx.sum() - xx.diag().sum()) / (N * (N - 1))
    yy_val = (yy.sum() - yy.diag().sum()) / (N * (N - 1))
    xy_val = xy.mean()

    return xx_val + yy_val - 2 * xy_val

def vae_loss_mmd(recon_x, x, z, device, lmbda=10000.0):
    recon   = F.mse_loss(recon_x, x, reduction="sum")
    z_prior = torch.randn_like(z).to(device)    # muestras de la prior
        N(0,I)
    mmd     = compute_mmd(z, z_prior)
    return recon + lmbda * mmd, recon, lmbda * mmd
```

La función `compute_kernel` construye la matriz de kernel completa ($N \times M$) expandiendo los tensores para evitar bucles explícitos. En `compute_mmd`, la diagonal de `xx` e `yy` se elimina antes de promediar porque representa la similitud de cada muestra consigo misma, que introduce sesgo en la estimación no centrada de la MMD.

Peso de la regularización: $\lambda = 10000$

La MMD produce valores numéricamente mucho más pequeños que la KL. Con $\lambda = 1$, el término de regularización sería despreciable frente al MSE y la red lo ignoraría por completo. Se escoge $\lambda = 10000$ para que la contribución de la MMD al gradiente sea comparable a la del error de reconstrucción, logrando un equilibrio análogo al que el VAE consigue de forma natural con $\beta = 1$.

2.2. Técnicas de regularización del entrenamiento

2.2.1. KL Annealing en el entorno 2D

Para evitar el colapso del espacio latente durante las primeras épocas de entrenamiento, se aplica *KL Annealing* [3]. El problema que motiva esta técnica es cuando el término KL tiene peso pleno desde el inicio, el optimizador encuentra rápidamente el mínimo trivial de enviar toda la varianza posterior a cero ($\sigma \to 0$, $\mu \to 0$), ignorando completamente la información de la entrada. El espacio latente queda colapsado y el decoder aprende a generar imágenes medias sin usar z.

La solución es arrancar con un peso $\beta_t = 0$ y subirlo linealmente hasta $\beta = 1$ a lo largo de los primeros `warmup_steps = 2000` pasos:

$$\beta_t = \text{mín}\left(1,\ \frac{t}{T_{\text{warmup}}}\right) \cdot \beta \tag{2.6}$$

Durante la fase de calentamiento el modelo se comporta como un autoencoder estándar, aprendiendo a reconstruir bien. Cuando el peso KL empieza a crecer, el espacio latente ya tiene estructura y el modelo puede regularizarlo sin colapsar. Esta técnica no se aplica al WAE-MMD porque la MMD es una distancia distribucional que no sufre del mismo problema de colapso.

2.2.2. Dropout y Early Stopping en el entorno 3D

En el modelado volumétrico tridimensional, la inmensa cantidad de parámetros que manejan las capas tridimensionales induce una fuerte tendencia a la memorización exacta de píxeles (*overfitting*). Para mitigar este comportamiento sin penalizar el cuello de botella distribucional, se incorporan técnicas alternativas de regularización estructural:

- **Dropout Estocástico (0.3):** Se intercalan capas de descarte aleatorio con un ratio del 30 % inmediatamente después de las activaciones de los bloques convolucionales y de la capa densa intermedia. Esto impide el co-ajuste de neuronas vecinas y fuerza a la red a extraer características geométricas globales del volumen.
- **Early Stopping (Parada Temprana):** Se monitoriza en tiempo real la métrica de pérdida de validación (`val_loss`). Si la métrica no disminuye de forma

significativa durante un intervalo de paciencia (*patience*) fijado en 5 épocas, el entrenamiento aborta de forma automática y recupera los mejores pesos guardados (`restore_best_weights=True`), aislando al modelo final de la fase de sobreajuste.

2.3. Arquitecturas encoder-decoder

Todos los modelos siguen una arquitectura convolucional simétrica. El encoder reduce espacialmente la entrada mediante capas convolucionales y proyecta la representación final a través de capas lineales al espacio latente. El decoder invierte este proceso con capas transpuestas.

2.3.1. Diferencia entre VAE y WAE en la arquitectura 2D

La única diferencia arquitectónica entre VAE y WAE reside en el encoder. El encoder del VAE produce dos salidas: μ y $\log \sigma^2$, necesarias para el truco de reparametrización. El encoder del WAE produce directamente un vector determinista z, ya que la estocásticidad no es necesaria cuando se usa MMD como regularizador. Para reflejar esto, se definen clases separadas para el encoder de cada variante:

Listing 2.4: Encoders VAE y WAE para MNIST (misma estructura convolucional, diferente cabeza lineal).

```
# Encoder VAE: salida mu + logvar (para reparametrizacion)
class MNISTEncoder(nn.Module):
    def __init__(self, latent_dim=32):
        super().__init__()
        self.net = nn.Sequential(
            nn.Conv2d(1, 32, 4, 2, 1),   # -> 16x16
            nn.BatchNorm2d(32), nn.LeakyReLU(0.2, True),
            nn.Conv2d(32, 64, 4, 2, 1),  # -> 8x8
            nn.BatchNorm2d(64), nn.LeakyReLU(0.2, True),
            nn.Conv2d(64, 128, 4, 2, 1), # -> 4x4
            nn.BatchNorm2d(128), nn.LeakyReLU(0.2, True),
        )
        self.fc_mu     = nn.Linear(128*4*4, latent_dim)
        self.fc_logvar = nn.Linear(128*4*4, latent_dim)

    def forward(self, x):
        h = self.net(x).view(x.size(0), -1)
        return self.fc_mu(h), self.fc_logvar(h)  # dos salidas

# Encoder WAE: salida directa z (sin reparametrizacion)
class MNISTEncoderWAE(nn.Module):
    def __init__(self, latent_dim=32):
        super().__init__()
        self.net = nn.Sequential(   # misma red convolucional
            nn.Conv2d(1, 32, 4, 2, 1),
            nn.BatchNorm2d(32), nn.LeakyReLU(0.2, True),
            nn.Conv2d(32, 64, 4, 2, 1),
            nn.BatchNorm2d(64), nn.LeakyReLU(0.2, True),
            nn.Conv2d(64, 128, 4, 2, 1),
            nn.BatchNorm2d(128), nn.LeakyReLU(0.2, True),
        )
```

```
        self.fc_z = nn.Linear(128*4*4, latent_dim)  # una sola salida

    def forward(self, x):
        h = self.net(x).view(x.size(0), -1)
        return self.fc_z(h)   # una sola salida
```

A nivel de modelo completo, la distinción entre MNISTVAE y MNISTWAE se refleja en el método forward: el VAE devuelve (recon, mu, logvar) y aplica el truco de reparametrización, mientras que el WAE devuelve (recon, z) directamente:

Listing 2.5: Modelos completos VAE y WAE para MNIST.

```
class MNISTVAE(nn.Module):
    def __init__(self, latent_dim=32):
        super().__init__()
        self.enc = MNISTEncoder(latent_dim)
        self.dec = MNISTDecoder(latent_dim)

    def reparameterize(self, mu, logvar):
        std = torch.exp(0.5 * logvar)
        eps = torch.randn_like(std)   # ruido ~ N(0,1)
        return mu + eps * std         # z ~ N(mu, sigma^2)

    def forward(self, x):
        mu, logvar = self.enc(x)
        z    = self.reparameterize(mu, logvar)
        recon = self.dec(z)
        return recon, mu, logvar      # 3 elementos

class MNISTWAE(nn.Module):
    def __init__(self, latent_dim=32):
        super().__init__()
        self.enc = MNISTEncoderWAE(latent_dim)
        self.dec = MNISTDecoder(latent_dim)

    def forward(self, x):
        z     = self.enc(x)
        recon = self.dec(z)
        return recon, z               # 2 elementos
```

Esta distinción en el número de elementos devueltos por forward es la que aprovechan las funciones de visualización para detectar automáticamente si están ante un VAE o un WAE, sin necesidad de pasar un argumento extra.

2.3.2. Adaptación a múltiples datasets 2D

Para CIFAR-10 las arquitecturas son análogas pero con entrada de 3 canales y dimensión latente de 128. El patrón encoder/decoder se replica exactamente. Las funciones de visualización detectan automáticamente el número de canales de la imagen reconstruida para elegir el mapa de color correcto (gray para MNIST/FashionMNIST, RGB para CIFAR-10).

2.3.3. Arquitectura Convolucional base para Volúmenes 3D

Para la primera aproximación al modelado de tensores volumétricos, se estableció una resolución de trabajo de $16 \times 16 \times 16 \times 1$ y una dimensión latente de $d = 32$. Se diseñaron funciones de construcción en TensorFlow utilizando convoluciones tridimensionales puras sin regularización estructural adicional:

Listing 2.6: Funciones de construcción de la arquitectura convolucional VAE 3D base (16x16x16).

```
def build_encoder(latent_dim=32):
    encoder_inputs = tf.keras.Input(shape=(16, 16, 16, 1))

    # Reducciones sucesivas
    x = layers.Conv3D(16, 3, activation="relu", strides=2, padding="
        same")(encoder_inputs) # -> 8x8x8
    x = layers.Conv3D(32, 3, activation="relu", strides=2, padding="
        same")(x)              # -> 4x4x4
    x = layers.Conv3D(64, 3, activation="relu", strides=2, padding="
        same")(x)              # -> 2x2x2

    shape_before_flattening = tf.keras.backend.int_shape(x)[1:]

    x = layers.Flatten()(x)
    x = layers.Dense(128, activation="relu")(x)

    z_mean = layers.Dense(latent_dim, name="z_mean")(x)
    z_log_var = layers.Dense(latent_dim, name="z_log_var")(x)
    z = layers.Lambda(sampling, name="z")([z_mean, z_log_var])

    return tf.keras.Model(encoder_inputs, [z_mean, z_log_var, z], name
        ="encoder"), shape_before_flattening

def build_decoder(shape_before_flattening, latent_dim=32):
    latent_inputs = tf.keras.Input(shape=(latent_dim,))
    unflatten_dim = np.prod(shape_before_flattening)

    x = layers.Dense(unflatten_dim, activation="relu")(latent_inputs)
    x = layers.Reshape(shape_before_flattening)(x)

    # Expansiones sucesivas
    x = layers.Conv3DTranspose(32, 3, activation="relu", strides=2,
        padding="same")(x)     # -> 4x4x4
    x = layers.Conv3DTranspose(16, 3, activation="relu", strides=2,
        padding="same")(x)     # -> 8x8x8

    decoder_outputs = layers.Conv3DTranspose(1, 3, activation="sigmoid
        ", strides=2, padding="same")(x) # -> 16x16x16

    return tf.keras.Model(latent_inputs, decoder_outputs, name="
        decoder")
```

2.4. Bucle de entrenamiento y encapsulación

2.4.1. Función de entrenamiento unificada 2D

Se implementa una única función train_model que sirve tanto para el VAE-KL co- mo para el WAE-MMD, seleccionando el camino adecuado mediante el parámetro loss_type. Esto centraliza la lógica del bucle de entrenamiento, el KL Annealing y el registro de métricas:

Listing 2.7: Fragmento clave de la función de entrenamiento unificada.

```
for x, _ in trainloader:
    x = x.to(device)

    if loss_type == 'kl':
        current_weight = min(1.0, global_step / warmup_steps) * weight
        recon, mu, logvar = model(x)
        loss, rec, reg = vae_loss_kl(recon, x, mu, logvar,
                                     beta=current_weight)
    elif loss_type == 'mmd':
        current_weight = weight # sin annealing en MMD
        recon, z = model(x)
        loss, rec, reg = vae_loss_mmd(recon, x, z, device,
                                      lmbda=current_weight)

    opt.zero_grad() ; loss.backward() ; opt.step()
    global_step += 1
```

En el paso MMD, se recalcula z desde model(x) in lugar de reutilizar el z de una pasada anterior, ya que MMD necesita que z y zprior sean muestras frescas del mismo batch para que la estimación sea válida.

2.4.2. Encapsulación e inferencia integrada en Keras (3D)

A diferencia del entorno funcional imperativo implementado en PyTorch para 2D, el modelo tridimensional se encapsula extendiendo de forma orientada a objetos la clase `tf.keras.Model`. Esto permite redefinir de manera nativa los métodos virtuales `train_step` y `test_step` gestionando de forma interna el cálculo de gradientes a través de `tf.GradientTape()` y la actualización manual de los rastreadores de métricas (`metrics`). Esta abstracción estructural facilita la conexión directa con callbacks complejos y automatizaciones del framework sin alterar el bucle principal.

2.5. Funciones de visualización e interpolación

2.5.1. Espacio latente en entornos 2D y 3D

Visualización en 2D (t-SNE)

Se implementan tres niveles de visualización del espacio latente, con complejidad creciente: 1. `plot_tsne_classes_universal`: proyecta los vectores μ (VAE) o z (WAE) de todo el conjunto de test con t-SNE y colorea cada punto según su clase real. Permite evaluar

si el espacio latente separa semánticamente las categorías. 2. `tsne_prior_samples_universal`: genera 1000 muestras de la prior $\mathcal{N}(0, I)$, las decodifica y las vuelve a codificar para obtener sus representaciones latentes, que se proyectan con t-SNE. Revela si el decoder es capaz de generar imágenes coherentes en todo el espacio prior, o si hay regiones vacías que producen imágenes sin sentido. 3. `tsne_with_recon_thumbnails_universal`: superpone miniaturas de las imá- genes reconstruidas sobre su posición en el mapa t-SNE. Para no saturar el gráfico, se seleccionan puntos espacialmente dispersos mediante la función auxiliar `pick_spread_points`, que impone una distancia mínima entre los puntos elegidos.

Visualización en 3D (PCA y Manifold Volumétrico)

Dada la alta dimensionalidad del espacio latente entrenado ($d = 256$), para el análisis visual se emplea el Análisis de Componentes Principales (**PCA**) reduciendo a 2 componentes principales. Para evaluar la densidad del espacio, la visualización se divide en un mapeo de dispersión (*scatter plot*) filtrando y pintando de forma aislada cada una de las 4 clases de ModelNet mediante una estructura de diccionario para generar leyendas textuales explícitas. Asimismo, se codifica un mapeo del *Manifold 3D* que divide el plano PCA en una cuadrícula regular de 4×4 y decodifica las coordenadas inversas para generar visualizaciones completas de la deformación geométrica intermedia.

2.5.2. Interpolación en el espacio latente

Interpolación de píxeles (2D)

La función `interpolate_universal` toma dos imágenes reales del conjunto de test, las codifica y realiza una interpolación lineal entre sus representaciones latentes z_1 y z_2:

$$z_\alpha = (1 - \alpha)z_1 + \alpha z_2, \qquad \alpha \in \{0, \tfrac{1}{7}, \tfrac{2}{7}, \ldots, 1\} \tag{2.7}$$

Cada z_α se decodifica para obtener la imagen intermedia. Si las transiciones son suaves y semánticamente coherentes (p.ej., el trazo de un dígito cambia gradualmente), indica que el espacio latente es continuo y que la prior cubre bien la región codificada.

Interpolación de vóxeles y Renderizado (3D)

En el dominio volumétrico, se calcula la transición lineal directamente entre los vectores latentes decodificados por el submodelo *Decoder*. Al tratarse de una salida matricial continua probabilística, para el renderizado tridimensional se aplica un filtro umbral de presencia estricto ($> 0{,}5$). Si la certidumbre de la red supera dicho umbral, el punto se transforma en un cubo sólido utilizando la proyección de dibujo de polígonos `ax.voxels()` de `matplotlib`, permitiendo observar la metamorfosis física y la aparición o desaparición de componentes estructurales de los muebles.

Capítulo 3

Apartado 2D: VAE y WAE en imágenes 2D

3.1. Hiperparámetros específicos del apartado 2D

En este apartado se aplican los siguientes hiperparámetros según el dataset:

Dataset	Canales	Dim. latente	Épocas
MNIST	1	32	20
FashionMNIST	1	32	20
CIFAR-10	3	128	50

Cuadro 3.1: Hiperparámetros del Apartado 2D por dataset.

Todas las imágenes se redimensionan a 32×32 píxeles. MNIST y FashionMNIST son originalmente de 28×28, por lo que se aplica `transforms.Resize(32)` en el pipeline de carga de datos. La normalización lleva los píxeles al rango $[-1, 1]$.

3.2. MNIST

3.2.1. VAE clásico (KL)

Reconstrucciones

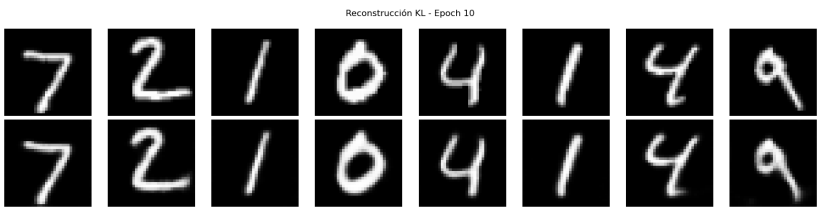


Figura 3.1: Reconstrucciones VAE-KL sobre MNIST (10 épocas).

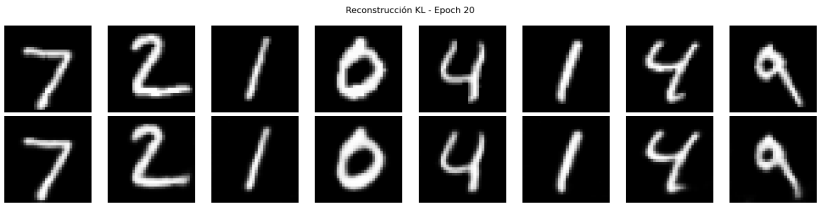


Figura 3.2: Reconstrucciones VAE-KL sobre MNIST (20 épocas).

Curvas de aprendizaje

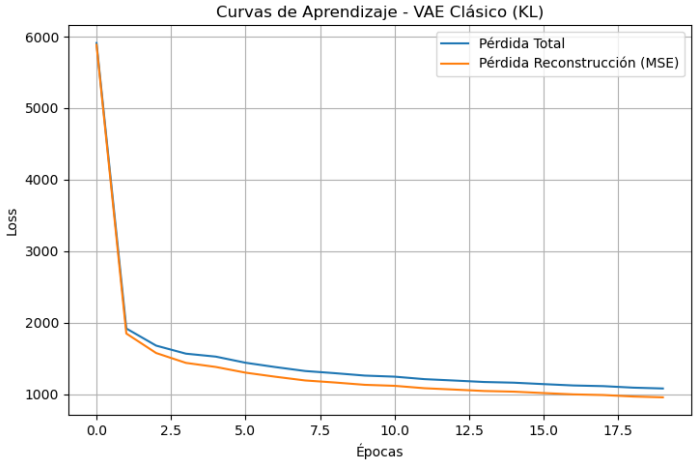


Figura 3.3: Curvas de aprendizaje VAE-KL en MNIST

Espacio latente (t-SNE)

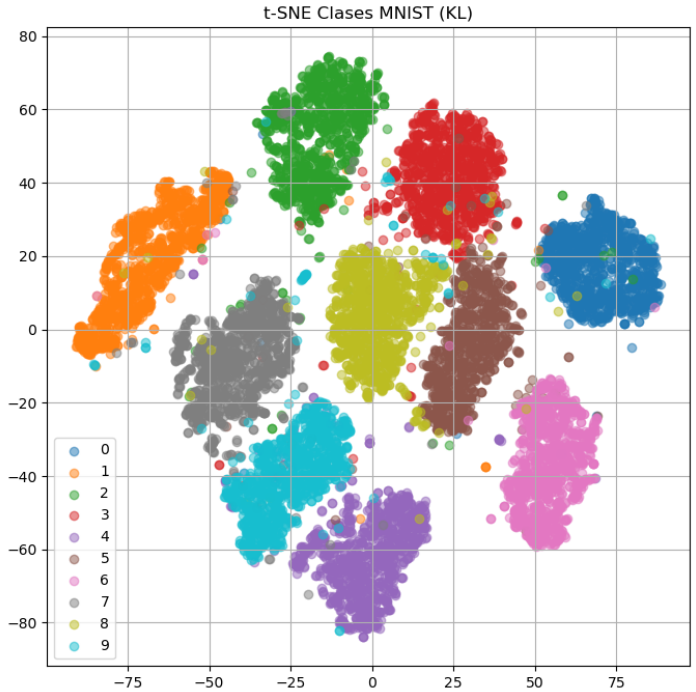


Figura 3.4: t-SNE del espacio latente VAE-KL en MNIST, coloreado por clase.

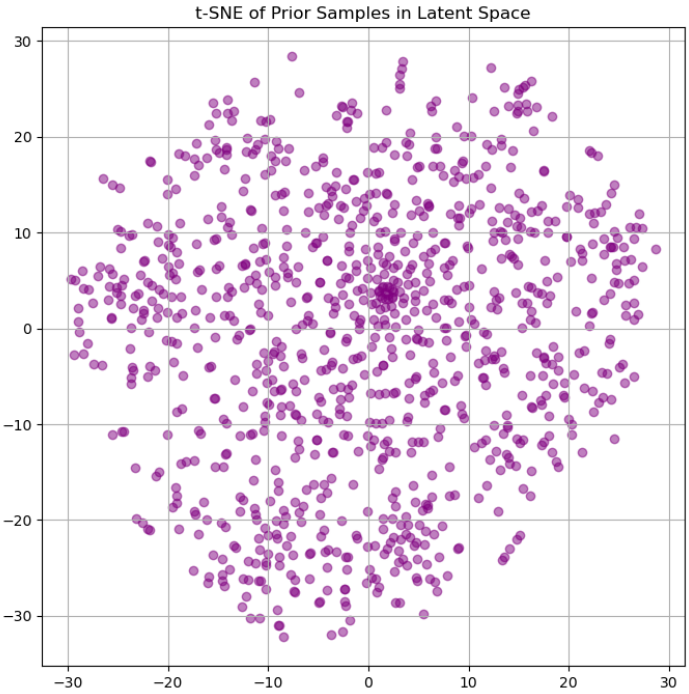


Figura 3.5: t-SNE del espacio latente VAE-KL en MNIST con muestras del prior.

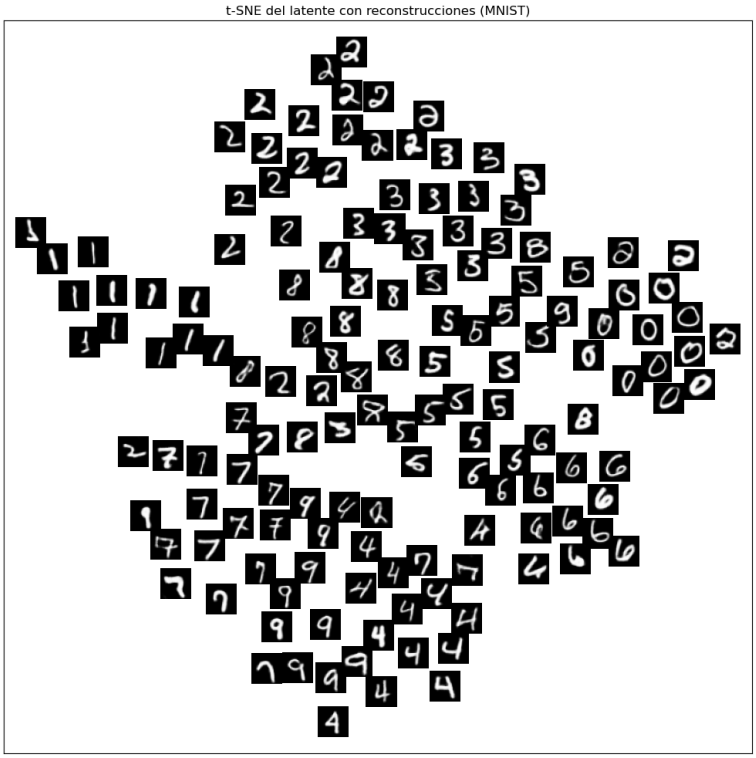


Figura 3.6: t-SNE del espacio latente VAE-KL con miniaturas de reconstrucción (MNIST).

Generación de nuevas imágenes

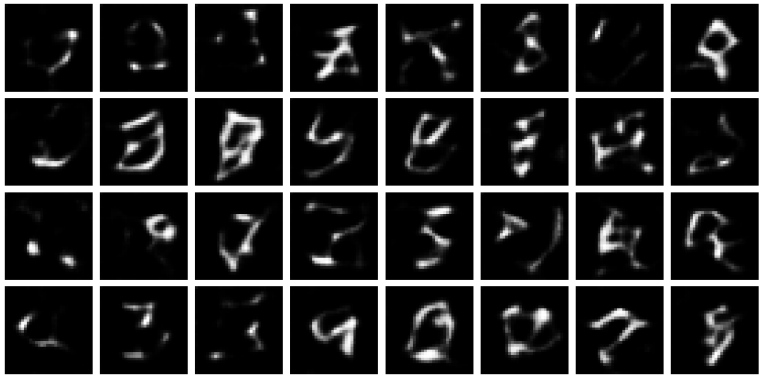


Figura 3.7: Muestras generadas por el VAE-KL entrenado en MNIST.

Interpolación en el espacio latente

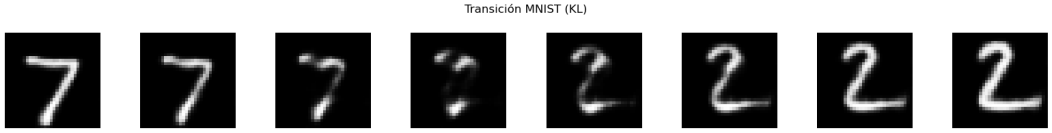


Figura 3.8: Interpolación lineal entre dos puntos del espacio latente (VAE-KL, MNIST).

3.2.2. WAE (MMD)

Reconstrucciones

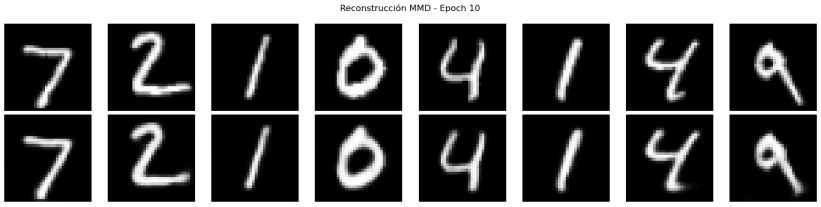


Figura 3.9: Reconstrucciones WAE-MMD sobre MNIST (10 épocas).

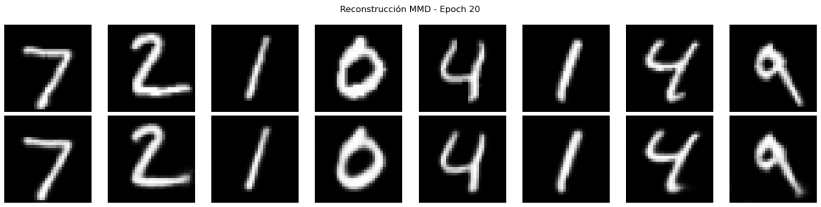


Figura 3.10: Reconstrucciones WAE-MMD sobre MNIST (20 épocas).

Curvas de aprendizaje

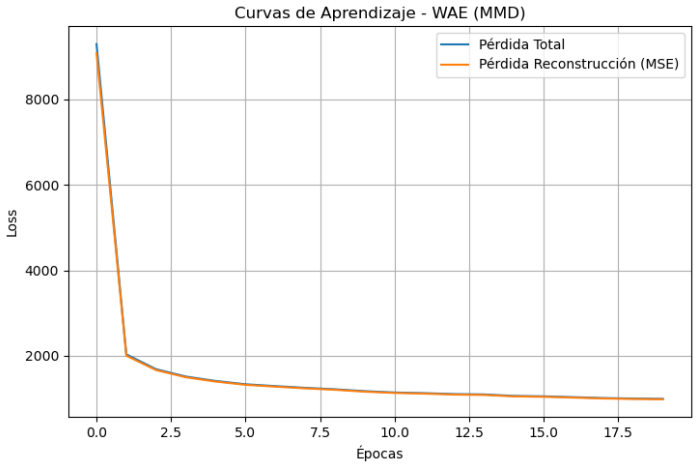


Figura 3.11: Curvas de aprendizaje WAE-MMD en MNIST

Espacio latente (t-SNE)

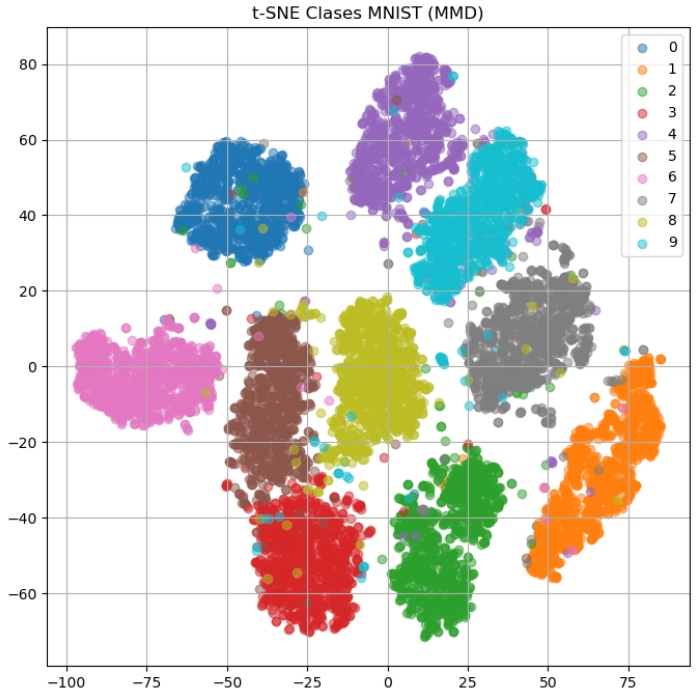


Figura 3.12: t-SNE del espacio latente WAE-MMD en MNIST, coloreado por clase.

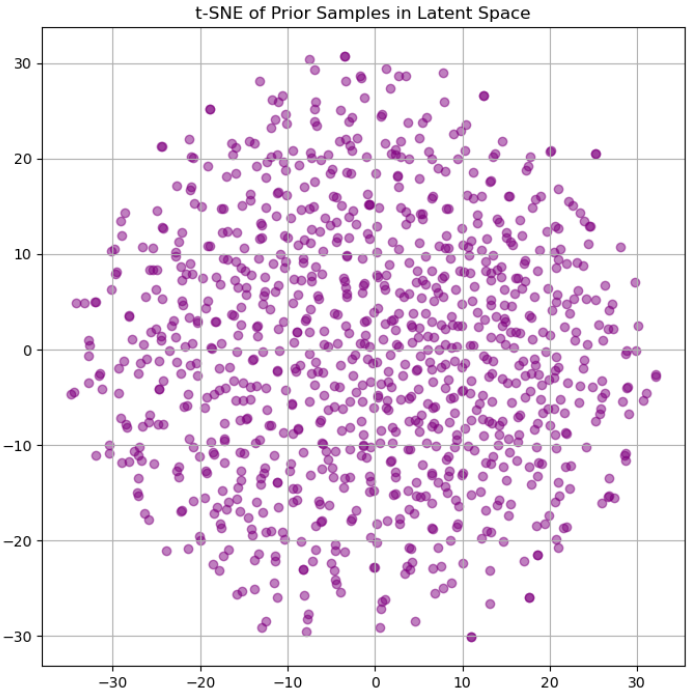


Figura 3.13: t-SNE del espacio latente WAE-MMD en MNIST con muestras del prior.

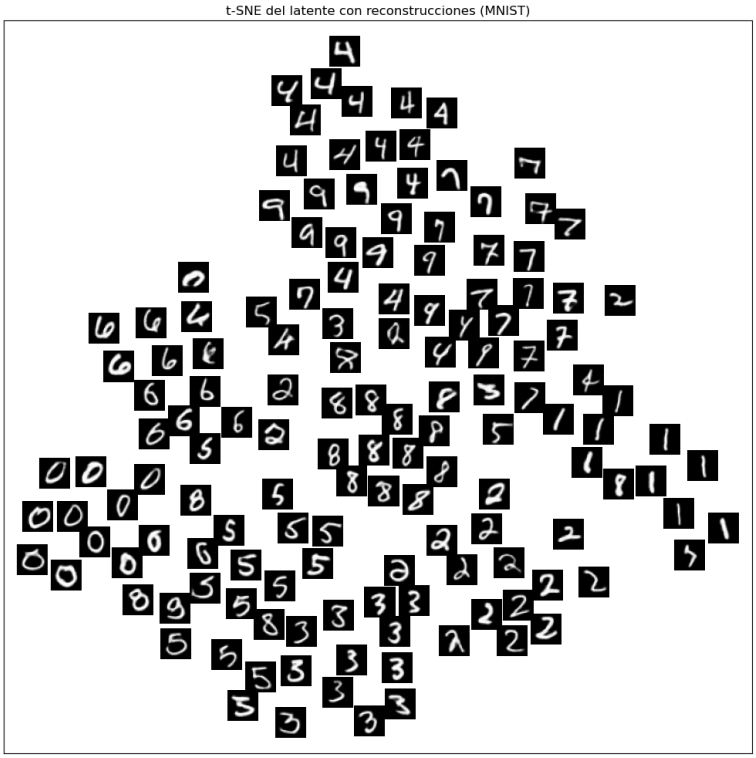


Figura 3.14: t-SNE del espacio latente WAE-MMD con miniaturas de reconstrucción (MNIST).

Generación de nuevas imágenes

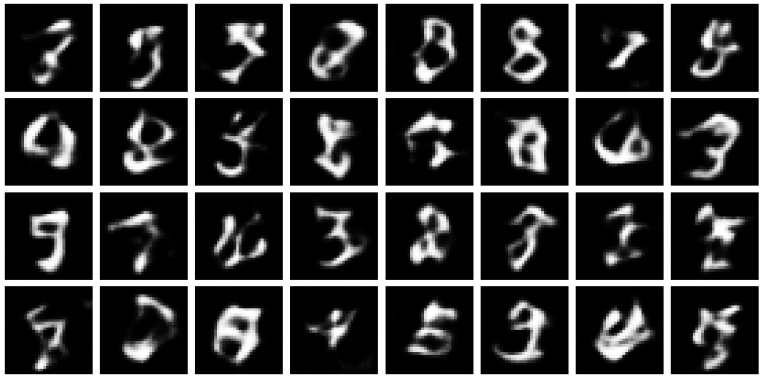


Figura 3.15: Muestras generadas por el WAE-MMD entrenado en MNIST.

Interpolación en el espacio latente



Figura 3.16: Interpolación lineal entre dos puntos del espacio latente (WAE-MMD, MNIST).

3.2.3. Evolución del entrenamiento

En lugar de listar todas las épocas, se recogen en la siguiente tabla los valores representativos de la función de pérdida en épocas clave:

	VAE-KL			**WAE-MMD**		
Época	Loss	Recon	KL	Loss	Recon	MMD
1	5911.2405	5886.1178	208.8152	9291.9960	9080.3640	211.6320
5	1524.4253	1379.7570	145.9063	1410.9431	1397.3792	13.5639
10	1258.8772	1129.9510	128.9262	1172.8482	1160.8174	12.0307
15	1159.8001	1034.2919	125.5081	1061.6995	1051.2422	10.4573
20	1078.6480	954.8857	123.7623	992.8742	982.7474	10.1268

Cuadro 3.2: Evolución de la pérdida durante el entrenamiento en MNIST.

3.2.4. Exploración de valores de peso

Peso (λ)	Métrica	Época 1	Época 5	Época 10	Época 15	Época 20
1000	Loss	7529.2	1386.1	1134.6	1020.3	946.1
	Recon	7444.8	1368.7	1129.3	1016.7	943.1
	MMD	84.4	17.3	5.3	3.6	2.9
5000	Loss	9186.2	1389.8	1139.6	1025.5	961.8
	Recon	9023.0	1379.9	1132.2	1018.6	955.5
	MMD	163.2	10.0	7.4	6.9	6.3
10000	Loss	9292.0	1410.9	1172.8	1061.7	992.9
	Recon	9080.4	1397.4	1160.8	1051.2	982.7
	MMD	211.6	13.6	12.0	10.5	10.1
20000	Loss	8204.0	1447.1	1211.1	1090.9	999.9
	Recon	7971.0	1424.7	1193.7	1074.3	987.1
	MMD	233.0	22.4	17.4	16.6	12.8
50000	Loss	9634.7	1523.3	1251.8	1133.2	1061.0
	Recon	9275.2	1479.2	1227.2	1112.0	1041.0
	MMD	359.4	44.1	24.6	21.2	19.9

Cuadro 3.3: Evolución de la pérdida del modelo WAE variando el hiperparámetro de regularización MMD (λ) en MNIST.

3.2.5. Análisis de la representación en MNIST

MNIST representa el escenario más sencillo de los tres evaluados: imágenes en escala de grises de dígitos aislados con baja variabilidad estructural. De esta manera, la arquitectura de variable latente logra converger y modelar la distribución de los datos sin mayores dificultades.

El análisis de las **curvas de aprendizaje** y los valores empíricos tabulados confirma la rápida convergencia de la arquitectura sobre este conjunto de datos. Durante las primeras cinco épocas, se observa una caída drástica en las métricas de error, indicando que el modelo asimila rápidamente la estructura topológica básica de los dígitos antes de entrar en la fase de ajuste más fino (época 10). Ambos modelos alcanzan finalmente valores de reconstrucción muy similares (en torno a 950-980 en la época 20), demostrando una capacidad equivalente para comprimir y descomprimir la información visual de MNIST.

Sin embargo, las gráficas revelan una diferencia fundamental en cómo cada modelo gestiona su regularización. En el VAE-KL, se aprecia una separación visible y constante entre la curva de la pérdida total y la de reconstrucción. La tabla respalda esta observación, ya que en la época 20, la divergencia KL mantiene un valor de 123.7. Para el VAE, la red asume un impuesto permanente en su función de pérdida para garantizar que las distribuciones latentes de cada muestra se ajusten estrictamente a la prior.

Por el contrario, la curva de aprendizaje del WAE-MMD muestra una superposición casi total entre la pérdida global y el error de reconstrucción. Los datos tabulados muestran

que la magnitud del término MMD es numéricamente marginal (apenas 10.1 en la época 20). Al penalizar únicamente la divergencia de la distribución agregada frente a la prior en lugar de forzar restricciones muestra a muestra, el WAE relaja la presión sobre el encoder. Esto permite que el optimizador destine la inmensa mayoría del gradiente a minimizar puramente el error de reconstrucción, estructurando el espacio latente sin meternos en una penalización numérica tan fuerte.

La exploración empírica del hiperparámetro λ (el peso del término MMD) ilustra claramente el compromiso (*trade-off*) entre la fidelidad de la reconstrucción y la regularización del espacio latente. Como se aprecia en la tabla, utilizar valores bajos de penalización ($\lambda \in \{1000, 5000\}$) minimiza notablemente el error de reconstrucción, pero reduce la fuerza con la que se alinea la distribución aprendida con la normal estándar (*prior*), provocando que la red tienda a comportarse como un autoencoder determinista clásico con menor capacidad generativa real. En el extremo opuesto, valores excesivamente restrictivos ($\lambda \in \{20000, 50000\}$) sobre-regularizan el espacio, lo que empieza a degradar la calidad de las imágenes recuperadas (elevando la pérdida de reconstrucción por encima de 1040). En consecuencia, la elección de $\lambda = 10000$ representa el punto de equilibrio óptimo: logra retener una reconstrucción altamente competitiva (982.7, manteniéndose en el mismo orden de magnitud que el VAE) a la vez que impone la estructura distribucional suficiente para garantizar un espacio latente continuo y bien cohesionado para las tareas de interpolación y generación libre.

En cuanto a la **calidad de reconstrucción**, los modelos son capaces de recuperar los números de manera excepcional. Como se observa en las figuras, los trazos se mantienen nítidos, se respeta el grosor y apenas hay pérdida de información, resultando en predicciones prácticamente perfectas de las imágenes originales. Esto demuestra que la dimensión del cuello de botella es más que suficiente para codificar la simplicidad topológica de los números sin sufrir un sobre-suavizado.

Al proyectar el **espacio latente** mediante t-SNE, se revela una estructura altamente organizada. Las representaciones se agrupan en clústeres sólidos y bien delimitados por clase numérica, indicando un fuerte aprendizaje semántico de forma puramente no supervisada. Las únicas zonas de posible solapamiento se dan de forma natural entre dígitos que comparten grandes similitudes (por ejemplo, las fronteras entre los números 4 y 9, o el 3 y el 8).

La **generación** libre a partir del espacio prior confirma que la red logra mapear correctamente el ruido hacia formas legibles. La gran mayoría de las muestras aleatorias son perfectamente reconocibles como números reales. No obstante, el decodificador produce ocasionalmente dígitos híbridos, trazos rotos o caracteres difusos.

Por último, la **interpolación** entre muestras de diferentes clases (como la transición ilustrada entre un 7 y un 2) muestra de manera continua la representación aprendida. En lugar de un cambio de opacidades a nivel de píxel, se observa una transformación geométrica: los trazos se acortan, se curvan y se recomponen de manera fluida.

3.3. FashionMNIST

3.3.1. VAE clásico (KL)

Reconstrucciones

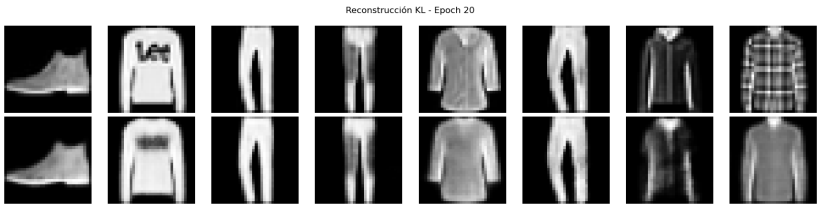


Figura 3.17: Reconstrucciones VAE-KL sobre FashionMNIST (20 épocas).

Espacio latente (t-SNE)

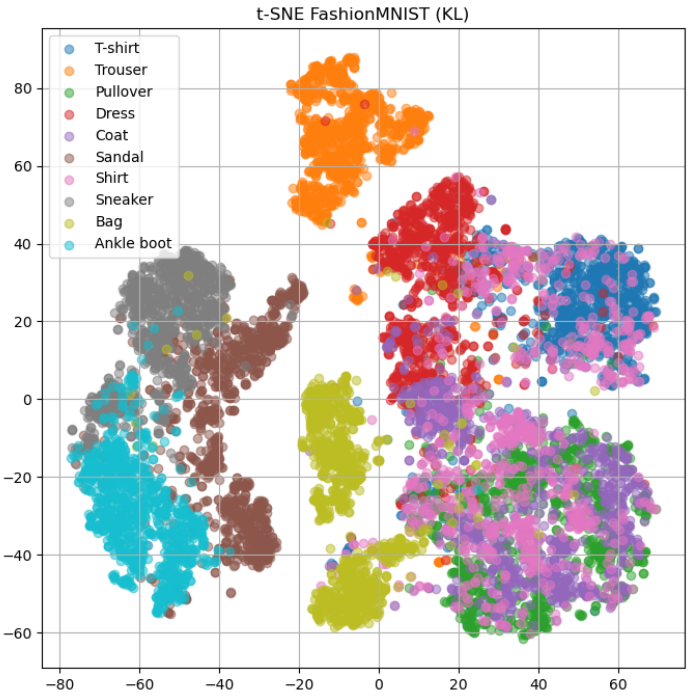


Figura 3.18: t-SNE del espacio latente VAE-KL en FashionMNIST, coloreado por clase.

Generación de nuevas imágenes



Figura 3.19: Muestras generadas por el VAE-KL entrenado en FashionMNIST.

Interpolación en el espacio latente

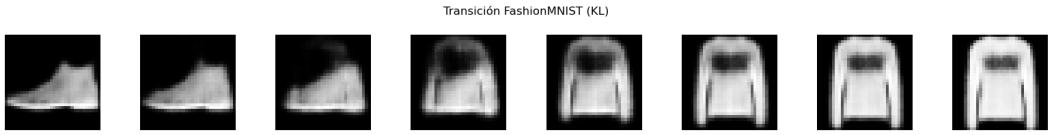


Figura 3.20: Interpolación lineal entre dos puntos del espacio latente (VAE-KL, FashionMNIST).

3.3.2. WAE (MMD)

Reconstrucciones

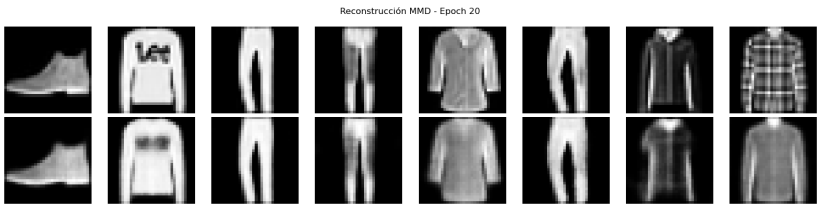


Figura 3.21: Reconstrucciones WAE-MMD sobre FashionMNIST (20 épocas).

Espacio latente (t-SNE)

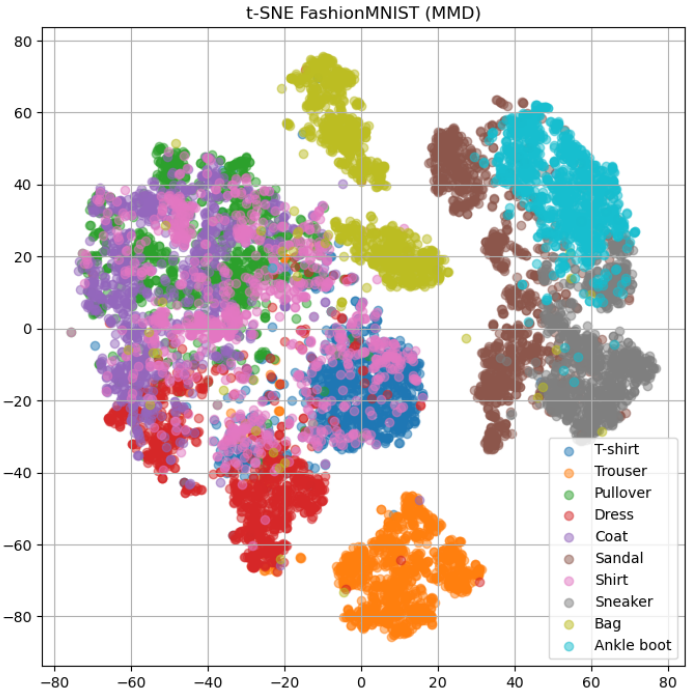


Figura 3.22: t-SNE del espacio latente WAE-MMD en FashionMNIST, coloreado por clase.

Generación de nuevas imágenes

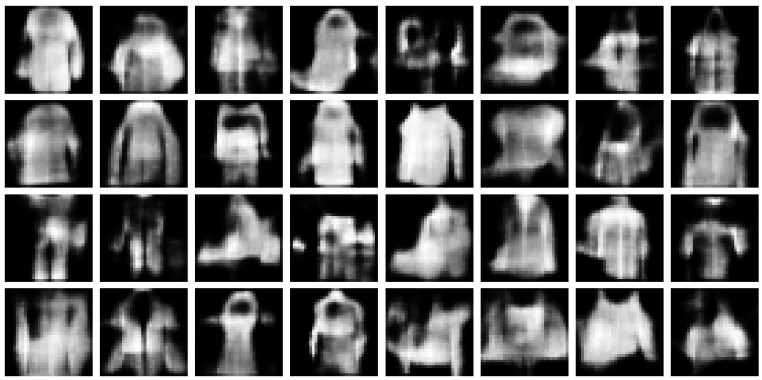


Figura 3.23: Muestras generadas por el WAE-MMD entrenado en FashionMNIST.

Interpolación en el espacio latente

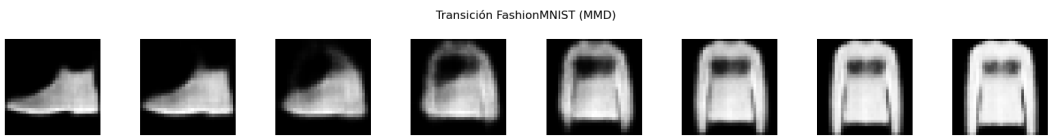


Figura 3.24: Interpolación lineal entre dos puntos del espacio latente (WAE-MMD, FashionMNIST).

3.3.3. Evolución del entrenamiento

	VAE-KL			WAE-MMD		
Época	Loss	Recon	KL	Loss	Recon	MMD
1	8180.4782	8155.8890	214.7022	8203.4746	7996.8413	206.6333
5	2644.5919	2505.4211	140.3546	2554.3958	2530.2147	24.1811
10	2314.0396	2187.9617	126.0779	2234.6346	2218.3202	16.3145
15	2178.4976	2056.5062	121.9913	2089.4835	2076.0207	13.4628
20	2099.5161	1979.1569	120.3592	2005.5063	1993.3801	12.1261

Cuadro 3.4: Evolución de la pérdida durante el entrenamiento en FashionMNIST.

3.3.4. Análisis de la representación en FashionMNIST

FashionMNIST supone un reto mayor que MNIST: aunque las imágenes siguen siendo en escala de grises y de resolución 28×28, la variabilidad intraclase es considerablemente mayor y hay clases con alta similitud visual.

En las tareas de **reconstrucción**, los modelos logran capturar adecuadamente la estructura global y la silueta para categorías con formas compactas y bien definidas (como zapatillas, pantalones o bolsos). Sin embargo, tienden a difuminar significativamente las prendas con detalle fino, como las texturas de los textos estampados en camisetas o los patrones de cuadros en camisas, resultando en un sobre-suavizado generalizado que evidencia las limitaciones de la compresión en el cuello de botella latente.

Al explorar el **espacio latente** mediante las proyecciones t-SNE, se observa una separación razonable entre las grandes supercategorías (el calzado y los bolsos forman clústeres bastante aislados del resto). No obstante, el principal desafío se encuentra en el denso solapamiento entre las distintas prendas de torso superior (camiseta, jersey, abrigo, camisa). Esto indica que la red tiene dificultades para separar características semánticas que visualmente comparten mucha información estructural.

La **generación** libre de nuevas muestras produce siluetas que resultan reconocibles como piezas de ropa, pero en ningún caso se alcanzan representaciones nítidas, confirmando que la captura de la distribución real de estas prendas requeriría arquitecturas generativas de mayor profundidad o resolución.

Finalmente, la **interpolación** entre dos prendas de clases radicalmente distintas (por ejemplo, la transición entre un zapato y un jersey) demuestra que el espacio latente aprendido es continuo, siendo una transición gradual sobre imágenes difusas que no se corresponden con ninguna prenda real hasta llegar a la forma final del objeto de destino.

3.4. CIFAR-10

3.4.1. VAE clásico (KL)

Reconstrucciones



Figura 3.25: Reconstrucciones VAE-KL sobre CIFAR-10 (50 épocas).

Espacio latente (t-SNE)

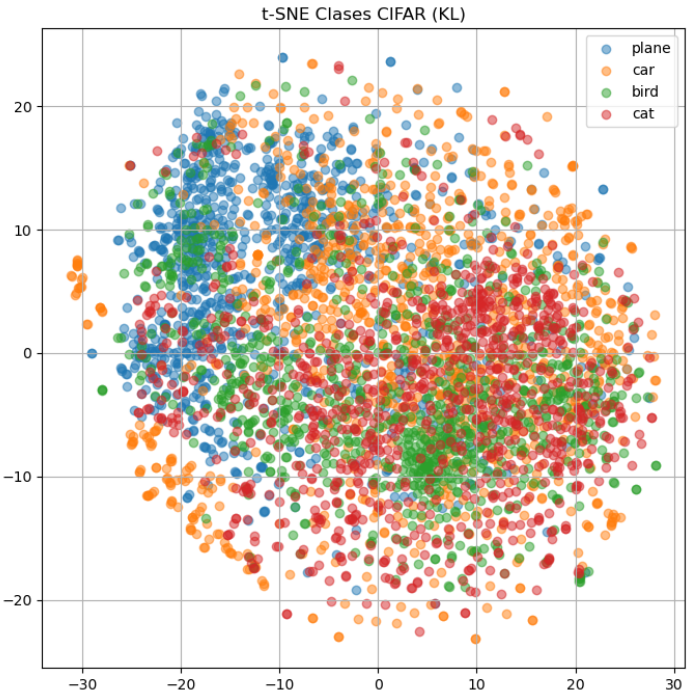


Figura 3.26: t-SNE del espacio latente VAE-KL en CIFAR-10, coloreado por clase.

Generación de nuevas imágenes



Figura 3.27: Muestras generadas por el VAE-KL entrenado en CIFAR-10.

Interpolación en el espacio latente



Figura 3.28: Interpolación lineal entre dos puntos del espacio latente (VAE-KL, CIFAR-10).

3.4.2. WAE (MMD)

Reconstrucciones



Figura 3.29: Reconstrucciones WAE-MMD sobre CIFAR-10 (50 épocas).

Espacio latente (t-SNE)

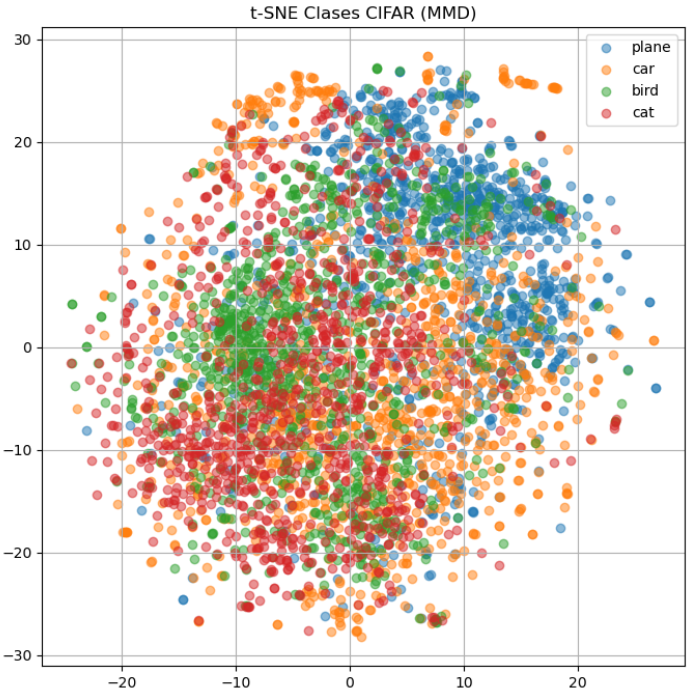


Figura 3.30: t-SNE del espacio latente WAE-MMD en CIFAR-10, coloreado por clase.

Generación de nuevas imágenes

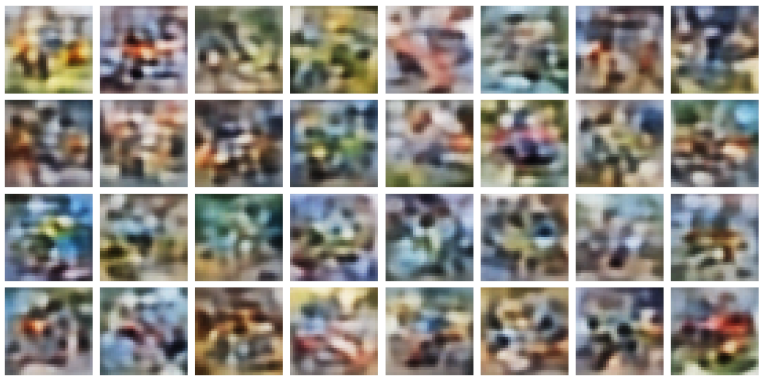


Figura 3.31: Muestras generadas por el WAE-MMD entrenado en CIFAR-10.

Interpolación en el espacio latente



Figura 3.32: Interpolación lineal entre dos puntos del espacio latente (WAE-MMD, CIFAR-10).

3.4.3. Evolución del entrenamiento

	VAE-KL			WAE-MMD		
Época	Loss	Recon	KL	Loss	Recon	MMD
1	44782.1843	44740.9752	3044.3023	41486.8925	41154.9728	331.9197
10	8868.7792	8457.4788	551.8871	8496.2125	8466.5728	29.6397
20	8257.6324	7801.3152	456.3172	7838.7736	7821.5249	17.2487
35	7764.3788	7331.9735	432.4053	7322.8117	7310.5302	12.2815
50	7528.9004	7104.9632	423.9372	7055.1488	7044.9761	10.1727

Cuadro 3.5: Evolución de la pérdida durante el entrenamiento en CIFAR-10.

3.4.4. Exploración del espacio de dimensiones

La dimensión del espacio latente es un hiperparámetro crítico en la arquitectura de los modelos generativos basados en autoencoders, ya que define directamente la capacidad del cuello de botella de información. Debido a los bajos resultados obtenidos en la ejecución anterior, en esta sección exploramos empíricamente el impacto de este parámetro entrenando nuestras redes con diferentes tamaños de representación latente. Vamos a probar con $d = 64$ y $d = 256$:

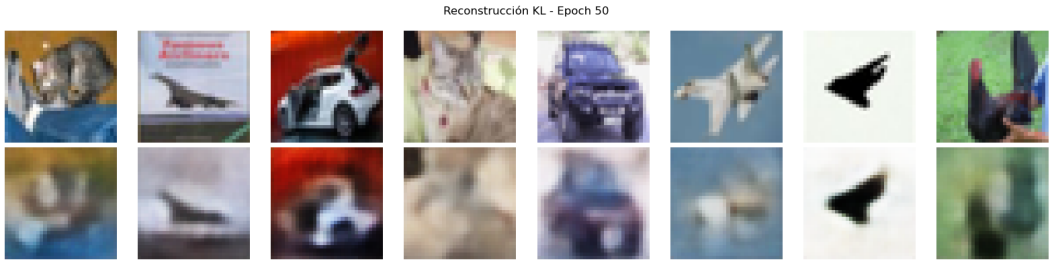


Figura 3.33: Reconstrucciones con latent_dim = 64 sobre CIFAR-10 (50 épocas).



Figura 3.34: Reconstrucciones con latent_dim = 256 sobre CIFAR-10 (50 épocas).

3.4.5. Análisis de la representación en CIFAR-10

CIFAR-10 es el dataset 2D más exigente de los 3 que hemos probado para el proyecto. Contiene imágenes RGB de 10 categorías semánticas muy distintas (animales, vehículos), con fondo variable, perspectivas múltiples y alta variabilidad intraclase. Con la arquitectura convolucional empleada y una dimensión latente de 128, los modelos se

ven incapaces de alcanzar reconstrucciones fotorrealistas, un resultado esperable dada la complejidad estructural y el cuello de botella de la red.

En las tareas de **reconstrucción**, los modelos logran capturar la paleta de colores dominante y la disposición espacial básica de las formas (por ejemplo, el cielo azul de fondo o la mancha roja de un coche), pero fracasan al intentar retener los detalles. Como se observa en las reconstrucciones, los contornos de los objetos aparecen extremadamente difusos y las texturas complejas (como el pelaje del gato o los detalles del avión) se pierden por completo. Esto refleja el fuerte efecto del cuello de botella latente, donde la red, incapaz de memorizar la alta frecuencia, se limita a generar una mancha de color que minimiza el error medio.

La proyección del **espacio latente** mediante t-SNE falla al realizar la separación entre clases. A diferencia de datasets más simples, las representaciones forman una nube densa con un solapamiento masivo entre categorías radicalmente distintas (*plane*, *car*, *bird*, *cat*). Esto demuestra que la red no ha logrado aprender características semánticas profundas que aíslen los conceptos visuales, limitándose probablemente a agrupar las imágenes por similitudes cromáticas o luminosidad global en lugar de por su contenido real.

En la **generación** de imágenes nuevas, se carece por completo de objetos reconocibles y se reducen a patrones borrosos y mezclas de colores que no poseen ninguna coherencia semántica ni estructura visual.

Respecto a la **dimensión latente**, los experimentos adicionales con $d = 64$ y $d = 256$ permiten observar que un cuello de botella demasiado estrecho ($d = 64$) degrada aún más la reconstrucción al no poder capturar la varianza de CIFAR, mientras que aumentarlo ($d = 256$) ofrece una mejora marginal a costa de un espacio latente más difícil de regularizar. La elección de $d = 128$ es la opción más equilibrada, aunque insuficiente para generar imágenes de calidad con esta profundidad de red.

Finalmente, la **interpolación** entre imágenes reales corrobora que el espacio latente es continuo, produciendo transiciones suaves de color y silueta. Sin embargo, al carecer de una buena organización semántica, los pasos intermedios se perciben como simples fundidos borrosos de distribuciones de color.

3.5. Conclusiones del Apartado 2D

A lo largo de este apartado se han entrenado y evaluado modelos VAE-KL y WAE-MMD sobre tres datasets de complejidad creciente. Las principales conclusiones son:

Calidad de reconstrucción. El WAE-MMD produce sistemáticamente reconstrucciones ligeramente más nítidas que el VAE-KL, especialmente en datasets con mayor variabilidad (FashionMNIST, CIFAR-10). La causa es estructural, ya que KL impone una distribución unimodal por muestra, lo que obliga al decoder a promediar, mientras que MMD no impone esa restricción a nivel de muestra individual.

Organización del espacio latente. El VAE-KL tiende a producir clústeres más esféricos y mejor separados por clase, favorecido por la presión que ejerce el ELBO sobre cada posterior individual. El WAE-MMD produce un espacio más uniforme a nivel global pero menos estructurado a nivel local de clase.

Calidad generativa. Ambos modelos son capaces de muestrear desde la prior y obtener imágenes plausibles en MNIST. En FashionMNIST y CIFAR-10 la calidad generativa decrece, siendo esta una limitación de la capacidad arquitectónica más que de la función de pérdida.

Efecto del KL Annealing. La técnica resulta esencial para el VAE-KL. Sin ella, el modelo converge al colapso del espacio latente en los primeros cientos de pasos. El calentamiento de 2000 pasos es suficiente para los tres datasets.

Escalabilidad con la complejidad de los datos. MNIST es trivial para ambos modelos; FashionMNIST empieza a revelar las limitaciones de la arquitectura convolucional de 3 bloques; CIFAR-10 las expone plenamente. Un paso natural de mejora sería mejorar la profundidad del encoder/decoder aumentando las convoluciones (a costa de realizar ejecuciones más pesadas).

Capítulo 4

Apartado 3D: Autoencoders en Volúmenes Tridimensionales

La segunda fase de la práctica explora la transición de las arquitecturas generativas del dominio 2D (imágenes planas de píxeles) al dominio 3D (volúmenes espaciales compuestos por vóxeles). Esta transición implica un crecimiento exponencial en la carga computacional, requiriendo convoluciones tridimensionales y un delicado ajuste de la memoria dinámica en las tarjetas gráficas (`set_memory_growth` en TensorFlow).

En esta sección abordaremos primero el conjunto de datos de números en 3D (3D MNIST), evaluando el comportamiento de nuestro modelo base VAE 3D.

4.1. Hiperparámetros específicos del apartado 3D

Para la primera aproximación al modelado de tensores volumétricos, se establecieron los siguientes hiperparámetros de trabajo:

Dataset	**Resolución / Canales**	**Dim. latente**	**Épocas**
3D MNIST (Números)	$16 \times 16 \times 16 \times 1$	32	50

Cuadro 4.1: Hiperparámetros del Apartado 3D para el modelo base.

Los datos fueron extraídos de un archivo HDF5 (`full_dataset_vectors.h5`). El último valor de la dimensión indica la presencia probabilística de materia en el vóxel. Dada la alta densidad de la pérdida de reconstrucción volumétrica (`binary_crossentropy` en los 3 ejes), se aplicó un factor de $\beta = 0{,}01$ para equilibrar la divergencia KL.

4.2. 3D MNIST (Dataset de Números en 3D)

4.2.1. VAE clásico 3D (KL)

Reconstrucciones Volumétricas (Evolución)

Para el renderizado de los volúmenes se codificó un *callback* visual que emplea `scipy.ndimage.zoom` para suavizar las predicciones y `ax.voxels` con un umbral de materia de 0,1.

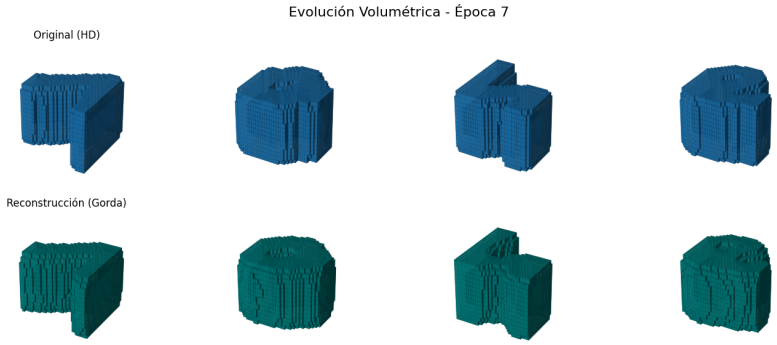


Figura 4.1: Evolución Volumétrica: Original vs Reconstrucción sobre 3D MNIST (10 épocas).

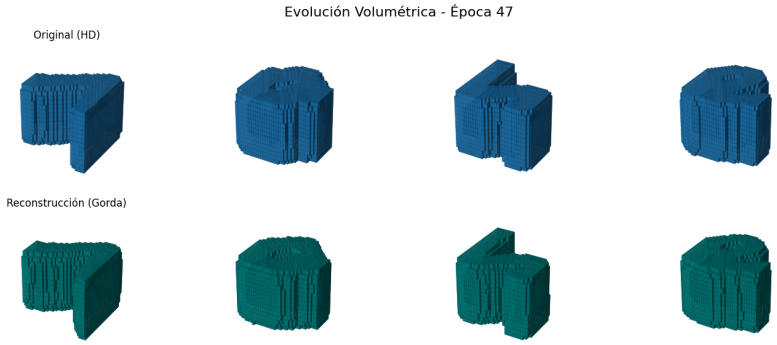


Figura 4.2: Evolución Volumétrica: Original vs Reconstrucción sobre 3D MNIST (50 épocas).

Comparativa Directa (Original vs. Reconstrucción)

Para apreciar con mayor precisión la fidelidad geométrica del modelo, se presenta a continuación una comparativa directa alineando los volúmenes reales (fila superior) con sus respectivas reconstrucciones (fila inferior), evidenciando la capacidad de la red para retener la topología del dígito.

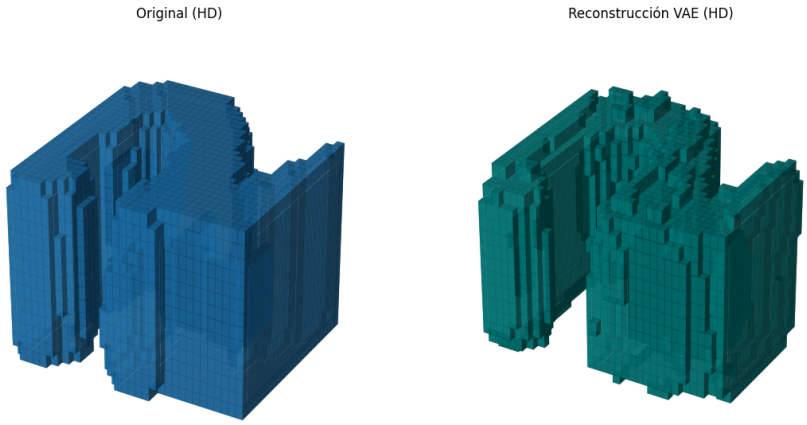


Figura 4.3: Comparativa directa real (arriba) vs reconstrucción (abajo) mediante el VAE-KL en 3D MNIST.

Curvas de aprendizaje



Figura 4.4: Curvas de aprendizaje del VAE 3D en el dataset 3D MNIST.

Espacio latente (PCA)

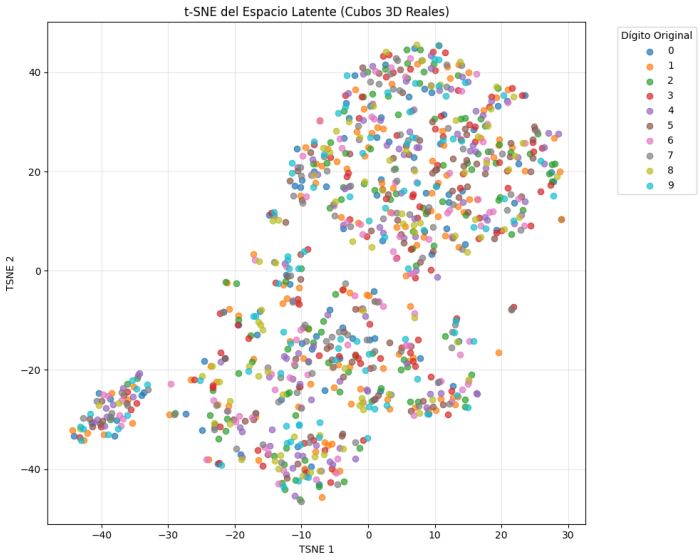


Figura 4.5: Proyección PCA del espacio latente VAE 3D en 3D MNIST, coloreado por dígitos.

Generación de nuevos volúmenes



Figura 4.6: Muestras volumétricas generadas a partir de ruido muestreado del prior $\mathcal{N}(0, I)$.

Interpolación en el espacio latente

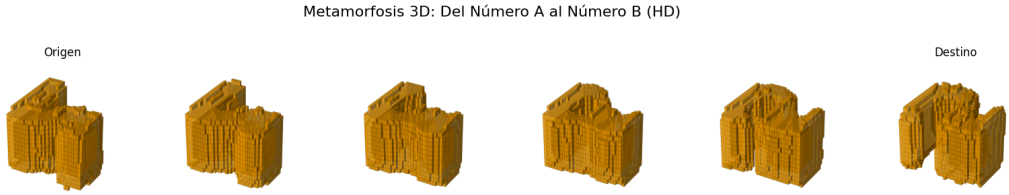


Figura 4.7: Metamorfosis volumétrica mediante interpolación lineal entre dos dígitos en 3D.

4.2.2. WAE 3D (MMD)

Reconstrucciones Volumétricas (Evolución)

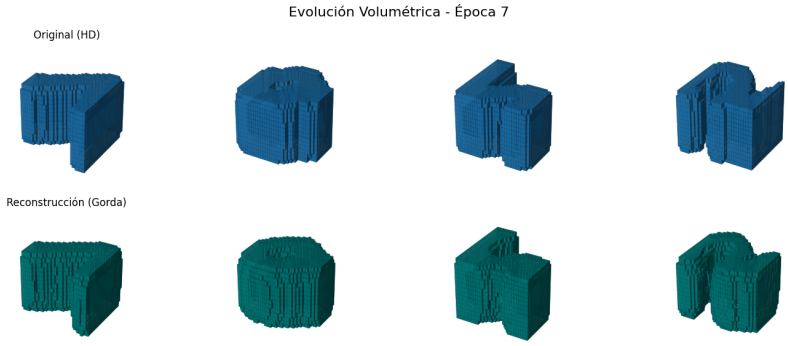


Figura 4.8: Evolución Volumétrica: Original vs Reconstrucción WAE-MMD sobre 3D MNIST (10 épocas).



Figura 4.9: Evolución Volumétrica: Original vs Reconstrucción WAE-MMD sobre 3D MNIST (50 épocas).

Comparativa Directa (Original vs. Reconstrucción)

Al igual que con el modelo KL, se expone la comparativa directa entre los volúmenes originales y las reconstrucciones generadas por el WAE-MMD para evaluar la nitidez relativa de los vóxeles.

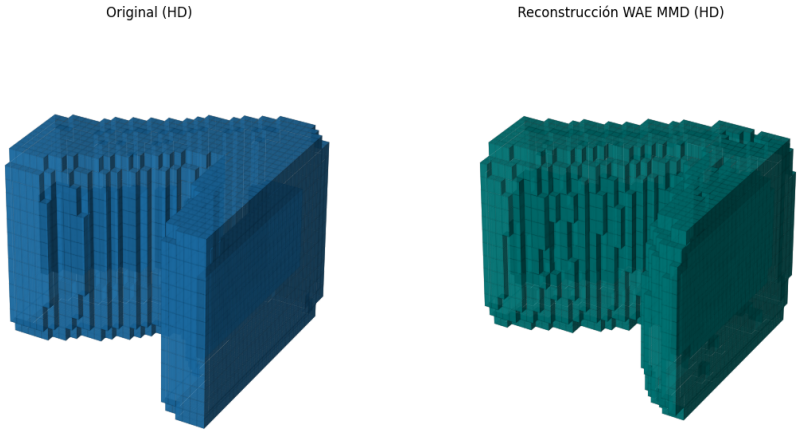


Figura 4.10: Comparativa directa real (arriba) vs reconstrucción (abajo) mediante el WAE-MMD en 3D MNIST.

Curvas de aprendizaje

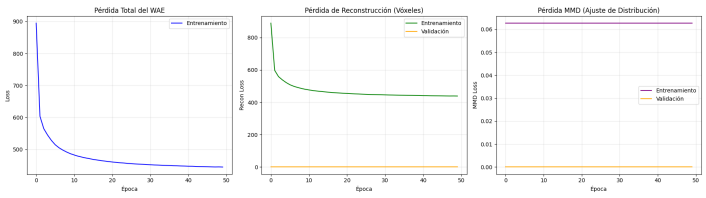


Figura 4.11: Curvas de aprendizaje del WAE 3D (MMD) en el dataset 3D MNIST.

Espacio latente (PCA)

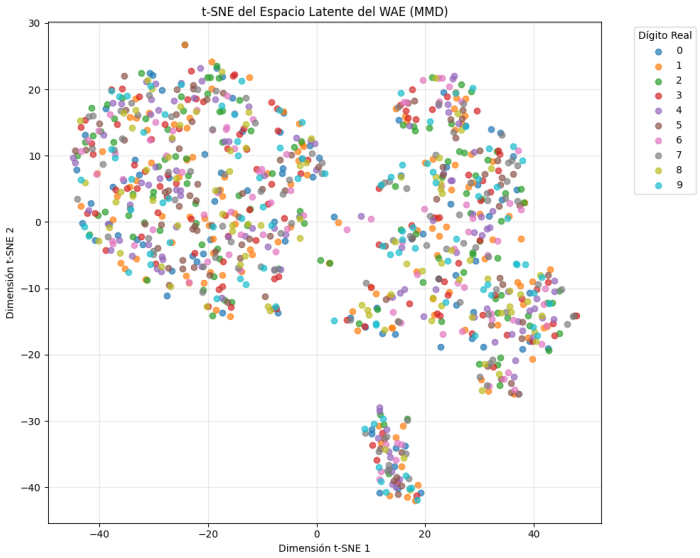


Figura 4.12: Proyección PCA del espacio latente WAE 3D en 3D MNIST, coloreado por dígitos.

Interpolación en el espacio latente

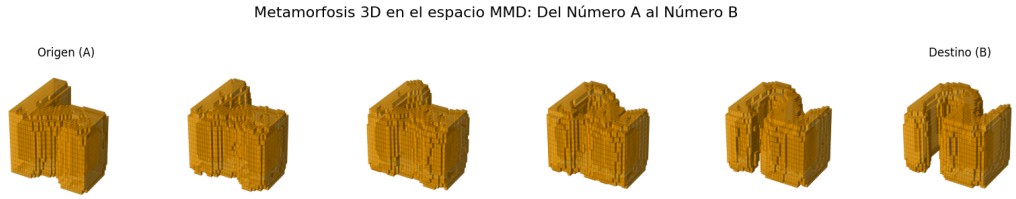


Figura 4.13: Metamorfosis volumétrica mediante interpolación lineal entre dos dígitos (WAE-MMD, 3D MNIST).

4.2.3. Evolución del entrenamiento

Se recogen en la siguiente tabla los valores representativos de la función de pérdida a lo largo del entrenamiento para comparar la convergencia de ambos modelos generativos:

	Train (Entrenamiento)			**Val (Validación)**		
Época	Total Loss	Recon	KL	Total Loss	Recon	KL
30 (Sin Regularizar)	*2749.46*	*2491.16*	*258.29*	*3818.52*	*3554.49*	*264.03*
1 (Modelo Definitivo)	9437.74	9421.41	16.32	7571.66	7471.88	99.78
10 (Modelo Definitivo)	4012.35	3821.12	191.23	4825.64	4621.10	204.54
Final (Parada Temprana)	2489.54	2218.31	271.23	3832.18	3574.60	257.58

Cuadro 4.2: Impacto del sobreajuste inicial y evolución del entrenamiento regularizado definitivo en ModelNet.

4.2.4. Análisis comparativo de la representación en 3D MNIST

El modelado de números en tres dimensiones presenta un desafío dual: la red debe aprender la topología de los trazos que forman el dígito mientras preserva la coherencia espacial y el "vacío"tridimensional que los rodea.

El análisis de las **curvas de aprendizaje** indica que ambas redes, a pesar de carecer de capas de regularización estructural severas (*Dropout*), logran estabilizar su función de pérdida. En el VAE, el uso de un factor $\beta = 0{,}01$ resulta crucial para evitar que la `binary_crossentropy` de los 16^3 vóxeles sofoque el gradiente KL. Por su parte, el WAE-MMD prescinde de la penalización muestra a muestra, lo que teóricamente debería permitir una caída más pronunciada en el error de reconstrucción, una tendencia que se confirma al observar cómo el modelo dedica mayor capacidad del gradiente a ajustar la topología del vóxel.

En cuanto a la **calidad de reconstrucción**, visible de forma detallada en las nuevas figuras de **Comparativa Directa**, ambos modelos son capaces de aproximar la volumetría original. Sin embargo, replicando el patrón observado en el Apartado 2D, el WAE-MMD produce superficies tridimensionales ligeramente más sólidas y definidas. El VAE-KL, condicionado por su naturaleza estocástica impuesta sobre cada variable latente individual, genera mayor "niebla de vóxeles"(difuminación espacial) alrededor de los trazos limpios.

La proyección del **espacio latente** refleja agrupaciones diferenciadas en ambos modelos. El VAE-KL fuerza clústeres más esféricos y localizados, lo que en $d = 32$ provoca áreas de superposición en los bordes de la distribución. El WAE-MMD distribuye los dígitos de manera más uniforme en el plano continuo, favoreciendo transiciones globales pero perdiendo algo de margen de separación local estricta entre clases muy similares geométricamente (como el 1 y el 7 en 3D).

En las pruebas de **generación** a partir de ruido prior, ambos modelos producen conglomerados de materia que asimilan la existencia de filamentos, si bien el WAE logra que estos "trazos ciegos"tengan un contorno ligeramente más cohesionado. Finalmente, la **interpolación** resulta un éxito en ambos enfoques: los vóxeles se reconfiguran físicamente en el espacio, estirando o contrayendo los trazos curvos para transformar suavemente un dígito en otro, confirmando que ambas funciones de pérdida logran organizar un hiperespacio topológicamente continuo a pesar de la maldición de la dimensionalidad.

4.3. ModelNet (Subconjunto de 4 clases)

El segundo bloque de experimentación tridimensional eleva la complejidad del problema al aplicar el modelo sobre formas poligonales complejas procedentes de un subconjunto de 4 clases del dataset **ModelNet**: *Bed* (Cama), *Chair* (Silla), *Monitor* e *Inodoro* (Toilet).

Dada la riqueza geométrica de los muebles, se escaló la resolución del volumen de forma drástica hasta un espacio cúbico de $32 \times 32 \times 32$ vóxeles y se expandió el cuello de botella latente hasta una dimensionalidad de $d = 256$.

4.3.1. El reto de la dimensionalidad: Overfitting y Regularización

El salto dimensional al volumen de 32^3 multiplicó de manera crítica la capacidad matemática de las capas densas y convolucionales del modelo. En una primera iteración exploratoria (sin regularización extra), el VAE exhibió un fenómeno agudo de **memorización pura (Overfitting)**, perdiendo la capacidad de generalizar a partir de la época 30.

Para solventar este problema y forzar a la red a aprender la semántica volumétrica en lugar de memorizar los píxeles, la implementación final introdujo dos técnicas combinadas:

1. **Dropout (0.3):** Aplicado tras las activaciones de los bloques convolucionales y densos.
2. **Early Stopping:** Con una paciencia de 5 épocas monitorizando de forma estricta la métrica `val_loss`.

La siguiente tabla contrasta el colapso del modelo inicial frente a la convergencia controlada del modelo regularizado definitivo:

	VAE-KL 3D (Validación)			**WAE-MMD 3D (Validación)**		
Época	Total Loss	Recon	KL	Total Loss	Recon	MMD
1	7571.66	7471.88	99.78	7136.56	6365.94	0.08
10	4825.64	4621.10	204.54	5055.09	4113.39	0.09
Final	3832.18	3574.60	257.58	4163.31	3655.25	0.05

Cuadro 4.3: Evolución comparativa de la pérdida de validación en ModelNet.

4.3.2. Resultados del VAE clásico 3D (KL)

A continuación, se presentan las visualizaciones y resultados extraídos única y exclusivamente del modelo final regularizado, el cual logró estructurar el espacio latente de manera exitosa.

Reconstrucciones Volumétricas

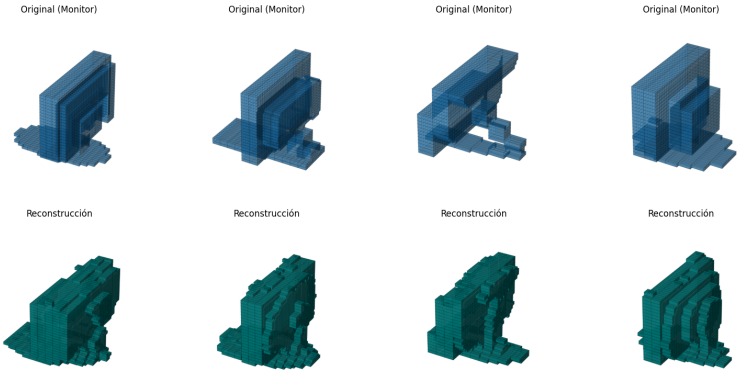


Figura 4.14: Reconstrucciones volumétricas en ModelNet bajo el VAE-KL (Época 10).

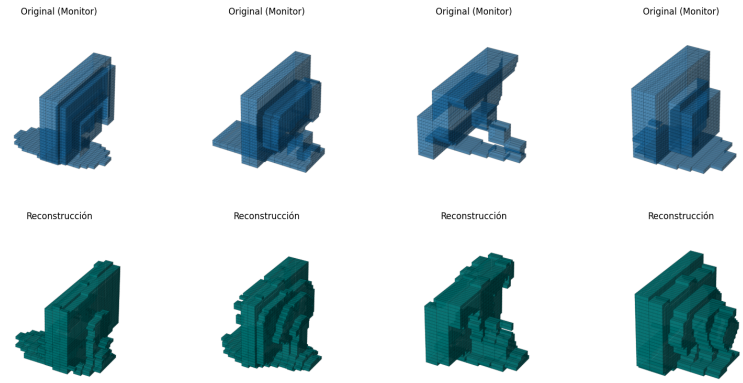


Figura 4.15: Reconstrucciones volumétricas finales optimizadas en ModelNet (VAE-KL).

Comparativa Directa (Original vs. Reconstrucción)

Debido a la mayor complejidad topológica de los muebles con respecto a los dígitos de MNIST, resulta esencial enfrentar visualmente el objeto real con el resultado del decodificador regularizado, observando directamente la retención de características estructurales.

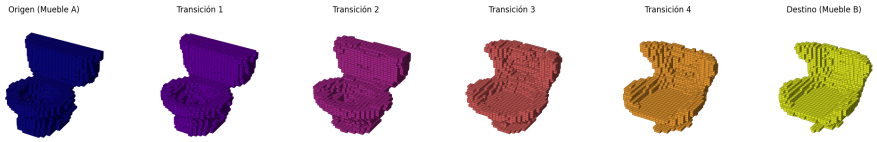


Figura 4.16: Comparativa directa real (arriba) vs reconstrucción (abajo) de muebles en ModelNet (VAE-KL).

Curvas de aprendizaje

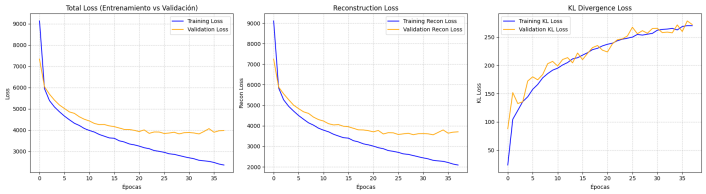


Figura 4.17: Curvas de aprendizaje del VAE-KL en ModelNet (nótese la estabilización de val_loss).

Espacio latente (PCA)

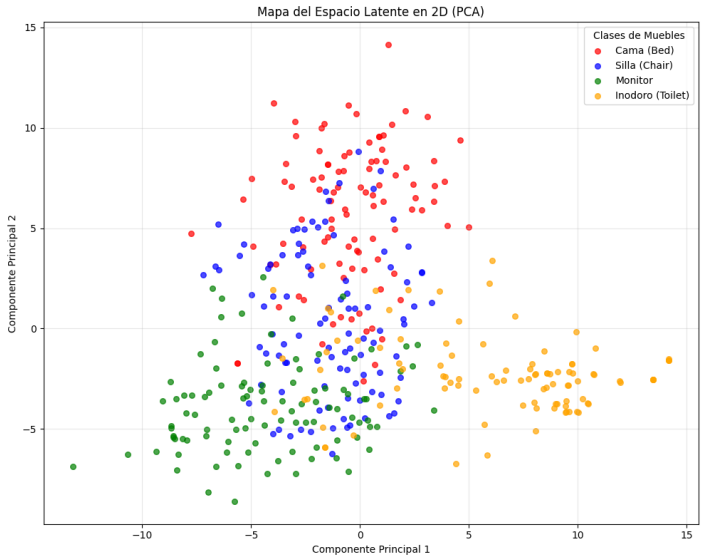


Figura 4.18: Proyección PCA del espacio latente estructurado de las 4 clases de muebles (VAE-KL).

Manifold del Espacio Latente y Generación libre

Para evaluar la continuidad de la distribución aprendida, se dividió el mapa del espacio latente en una cuadrícula regular de 4×4, muestreando puntos intermedios inversos para obligar al decodificador a esculpir las geometrías de transición.

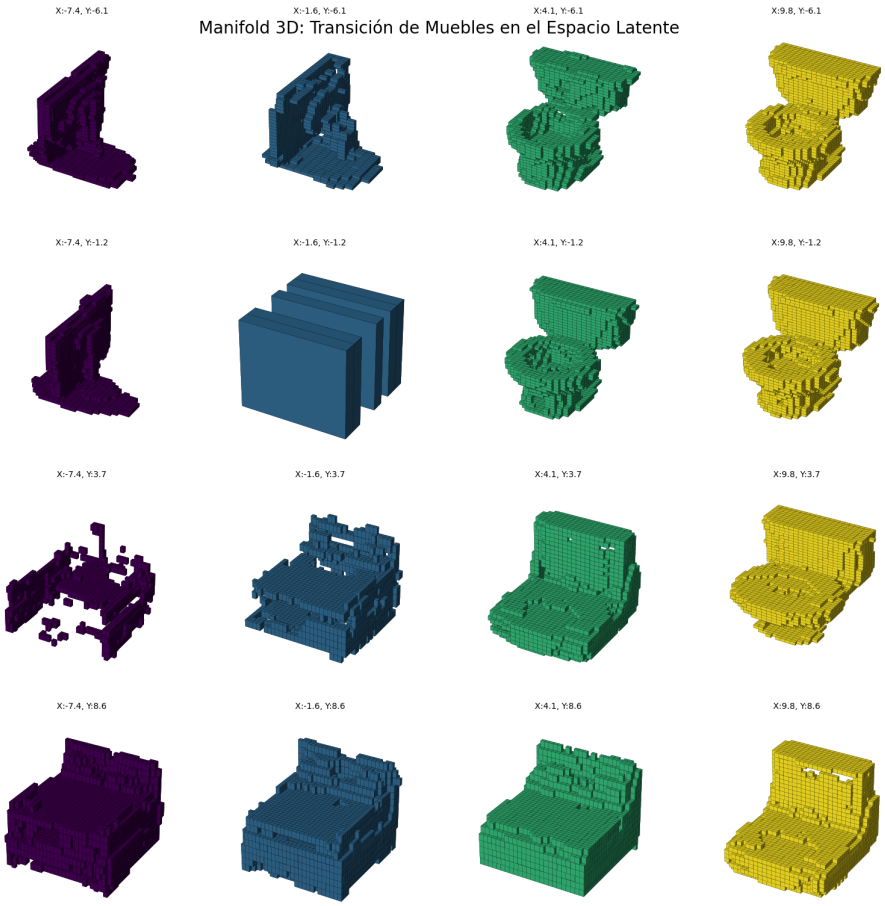


Figura 4.19: Manifold 3D: Cuadrícula 4×4 de transiciones geométricas continuas entre las familias de muebles.



Figura 4.20: Objetos volumétricos tridimensionales generados puramente a partir de ruido prior $\mathcal{N}(0, I)$.

Interpolación en el espacio latente

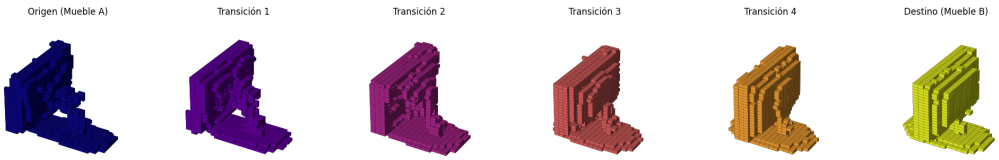


Figura 4.21: Metamorfosis geométrica continua mediante interpolación lineal entre dos muebles de ModelNet.

Análisis de la representación en ModelNet (VAE-KL)

El modelado tridimensional de objetos complejos expone las limitaciones y virtudes de la compresión en variables latentes estocásticas.

El análisis empírico revela que la inyección de la regularización estructural fue el factor determinante para estabilizar las métricas. Antes de aplicar Dropout, el espacio latente se encontraba topológicamente roto: al evaluar la cuadrícula del Manifold 3D, el decodificador caía en .ºcéanos vacíos"donde la incertidumbre era tan elevada que ningún vóxel superaba el umbral crítico de filtrado ($> 0{,}5$), resultando en coordenadas completamente vacías (ej. la coordenada intermedia X:11.6, Y:6.9 en las pruebas iniciales). Tras forzar al optimizador a extraer descriptores globales, las lagunas de información desaparecieron, logrando un *manifold* 3D densamente poblado y coherente (como se observa en la figura correspondiente).

En cuanto a la **calidad de reconstrucción** (evaluable en la *Comparativa Directa*), el VAE-KL regularizado logra aproximar con éxito la silueta física de los muebles de test, identificando componentes clave como las patas de las sillas, las bases de las camas o las pantallas de los monitores. No obstante, condicionado por la restricción probabilística impuesta, el decodificador genera "nubes difusas.[en] las fronteras de los objetos, sacrificando aristas completamente sólidas en favor de un espacio latente suave. Esto permite que, durante la **interpolación**, la transición volumétrica sea verdaderamente continua: los vóxeles no desaparecen abruptamente, sino que la masa geométrica se desplaza y reconfigura de forma orgánica para mutar de un concepto a otro.

4.3.3. Resultados del WAE 3D (MMD) Definitivo

Al igual que con el VAE, se procedió a entrenar la variante WAE-MMD utilizando la misma arquitectura regularizada (con Dropout y Early Stopping), sustituyendo la restricción KL por la minimización de la discrepancia MMD mediante el Kernel IMQ.

Reconstrucciones Volumétricas y Comparativa Directa

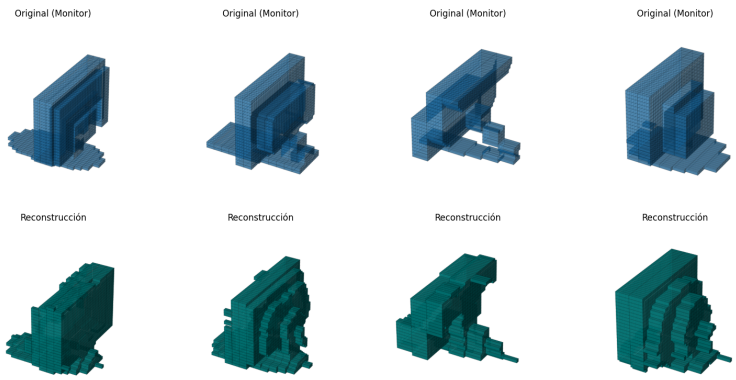


Figura 4.22: Reconstrucciones volumétricas finales optimizadas en ModelNet (WAE-MMD).

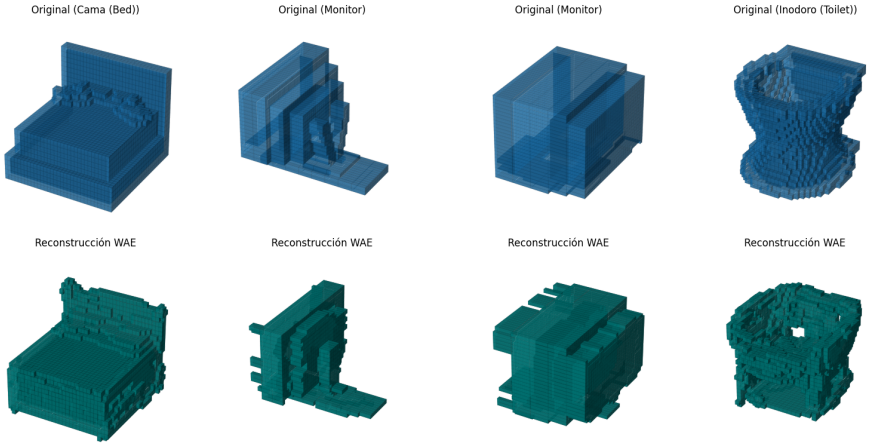


Figura 4.23: Comparativa directa real (arriba) vs reconstrucción (abajo) de muebles en ModelNet (WAE-MMD).

Curvas de aprendizaje

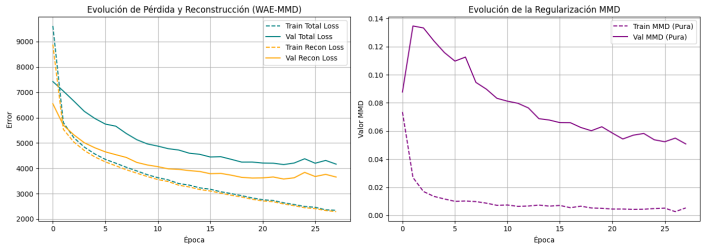


Figura 4.24: Curvas de aprendizaje del WAE-MMD en ModelNet.

Espacio latente (PCA)

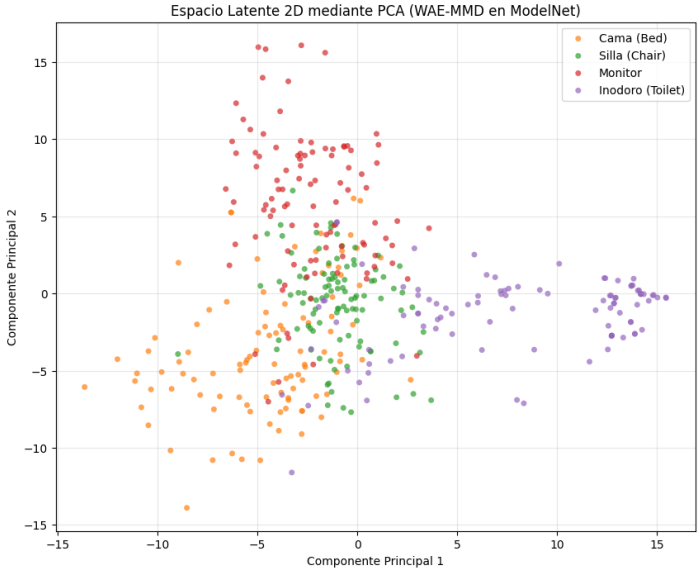


Figura 4.25: Proyección PCA del espacio latente de muebles (WAE-MMD).

Manifold 3D, Generación e Interpolación

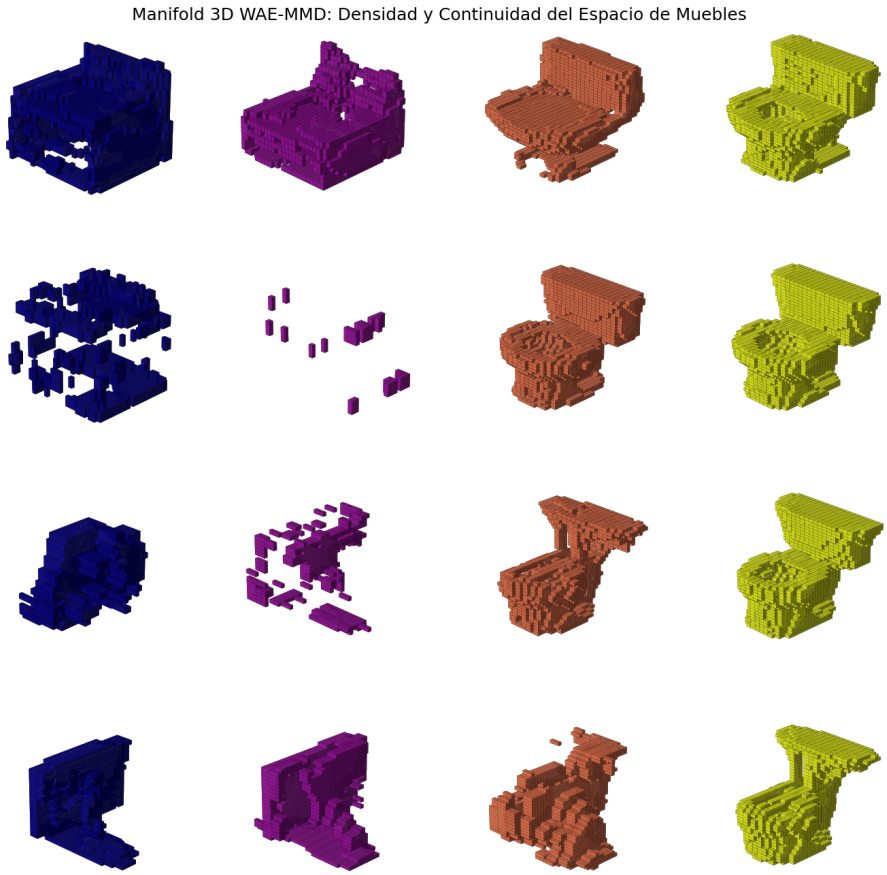


Figura 4.26: Manifold 3D: Cuadrícula 4×4 de transiciones geométricas continuas (WAE-MMD).

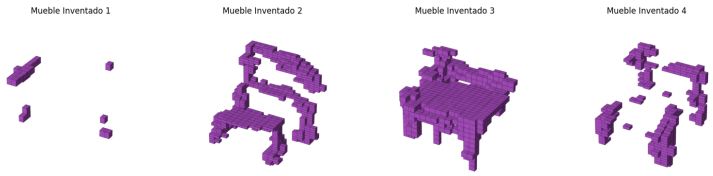


Figura 4.27: Objetos volumétricos tridimensionales generados a partir de ruido prior (WAE-MMD).

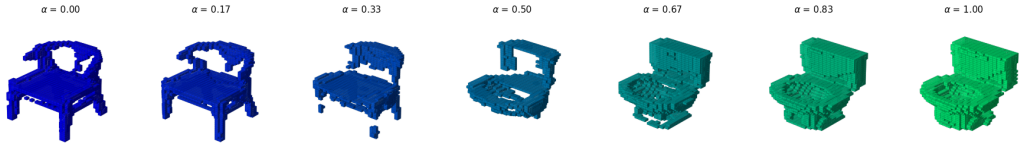


Figura 4.28: Metamorfosis geométrica continua mediante interpolación lineal (WAE-MMD).

4.3.4. Análisis comparativo de la representación en ModelNet (KL vs MMD)

En la siguiente tabla se compara la evolución de las métricas de validación entre ambos modelos regularizados para el dataset ModelNet, evaluando su capacidad real de generalización sobre muebles no vistos durante el entrenamiento:

	VAE-KL 3D (Validación)			**WAE-MMD 3D (Validación)**		
Época	Total Loss	Recon	KL	Total Loss	Recon	MMD
1	7571.66	7471.88	99.78	7136.56	6365.94	0.08
10	4825.64	4621.10	204.54	5055.09	4113.39	0.09
Final	3832.18	3574.60	257.58	4163.31	3655.25	0.05

Cuadro 4.4: Evolución comparativa de la pérdida de validación en ModelNet.

Las pruebas sobre el modelado tridimensional de objetos complejos confirman las hipótesis extraídas en el entorno bidimensional. El **WAE-MMD**, al eludir la restricción analítica de la divergencia KL sobre cada muestra y optimizar la distancia entre las distribuciones latentes globales mediante el Kernel IMQ, es capaz de dedicar una mayor fracción de su gradiente a minimizar el error de reconstrucción volumétrico (`binary_crossentropy`).

Visualmente, esto se traduce en que el WAE esculpe modelos con contornos topológicos más definidos (las patas de las sillas o los respaldos aparecen más sólidos y conectados) en comparación con las reconstrucciones del VAE, las cuales presentan mayor dispersión probabilística o "vóxeles flotantes.[en] las zonas de incertidumbre.

Sin embargo, en las proyecciones del espacio latente (**PCA**) y en las **interpolaciones**, el VAE demuestra una organización clúster ligeramente más definida y esférica, lo que hace que las transiciones entre un mueble y otro resulten marginalmente más fluidas y orgánicas, en contraste con el WAE, que tiende a priorizar la fidelidad del objeto final sacrificando cierta densidad en las zonas intermedias del *manifold*.

Capítulo 5

Conclusiones Generales

La presente práctica ha permitido realizar una inmersión completa en la formulación, implementación y análisis empírico de modelos generativos basados en el paradigma de autoencoders. A través de la experimentación metódica con el VAE clásico (regido por la divergencia KL) y el WAE (regido por la MMD), escalando la complejidad desde el dominio bidimensional de los píxeles hasta el entorno tridimensional de los vóxeles, podemos extraer las siguientes conclusiones fundamentales:

1. **Restricciones Estructurales vs. Fidelidad Generativa (KL frente a MMD):** Hemos comprobado de forma reiterada, tanto en MNIST, FashionMNIST, CIFAR-10 como en ModelNet, que la divergencia KL impone una regularización estocástica muy severa a nivel individual. Esto fuerza al espacio latente a mantener una topología altamente continua y clústeres de morfología esférica, pero obliga al decodificador a promediar y difuminar los resultados (imágenes borrosas o "niebla de vóxeles"). Por su parte, la MMD relaja esta presión, operando sobre las distribuciones agregadas, lo que permite que el modelo optimice de forma más agresiva el error de reconstrucción y genere objetos y contornos visualmente más definidos.

2. **La Maldición de la Dimensionalidad y el Overfitting:** El salto del entorno 2D al modelado volumétrico 3D ($32 \times 32 \times 32$) ha puesto de manifiesto la crítica explosión de parámetros en las redes neuronales profundas. En espacios dimensionalmente ricos, los modelos tienden inevitablemente a la memorización de los datos de entrenamiento (sobreajuste), rompiendo.[el] espacio latente continuo. El uso de técnicas de regularización agresivas, como el descarte aleatorio de pesos (*Dropout*) en capas profundas y la detención automática del entrenamiento (*Early Stopping*), se ha revelado como un requisito indispensable, más que como una simple buena práctica, para lograr que la red abstraiga conceptos geométricos y pueble el *manifold* de forma densa.

3. **Capacidad Arquitectónica y el Cuello de Botella:** Finalmente, los resultados difusos al reconstruir texturas complejas en CIFAR-10 y la falta de detalle en algunos muebles de ModelNet evidencian los límites naturales de compresión de información de los autoencoders de 3 o 4 bloques convolucionales. Si bien hemos demostrado que un cuello de botella latente dimensionado adecuadamente ($d = 128$ en 2D y $d = 256$ en 3D) logra organizar de forma no supervisada la información

semántica, alcanzar resultados fotorrealistas u objetos tridimensionales de alta fidelidad arquitectónica requeriría escalar radicalmente la profundidad del modelo, introducir arquitecturas con mecanismos de atención, o recurrir al actual estado del arte dominado por los modelos difusionales (*Diffusion Models*).

Bibliografía

[1] Kingma, D.P. & Welling, M. (2013). *Auto-Encoding Variational Bayes*. arXiv:1312.6114.

[2] Tolstikhin, I., Bousquet, O., Gelly, S., & Schölkopf, B. (2018). *Wasserstein Auto-Encoders*. ICLR 2018. arXiv:1711.01558.

[3] Bowman, S.R. et al. (2015). *Generating Sentences from a Continuous Space*. arXiv:1511.06349. [KL Annealing]

[4] LeCun, Y. et al. (1998). *Gradient-Based Learning Applied to Document Recognition*. Proceedings of the IEEE.